

# Data Mining 2026 — Project Notebook

---

**Course:** Data Mining

**Project track:** ☒ Standard Analysis ☐ Research-Oriented

**Group members:**

- Tereza Knápková
- Mária Matušisková
- Dominik Kysel
- Dominik Ott

**Dataset:**

- Bike Sharing Dataset (<https://archive.ics.uci.edu/dataset/275/bike+sharing+dataset>)

**Initial task description (Module 1 perspective):**

RESEARCH QUESTIONS:

- RQ1: Which temporal and weather factors are most associated with hourly bike demand ( cnt )?
- Before (expectations):
  - We expect that factors like time (hr) and temperature will be strongly related to demand, similarly registered and casual users, suspect that features like humidity, weathersit and season might have mild association with demand.
- After (our analysis):
  - **Top Associated Factors (by correlation strength):** registered ( $r=0.972$ ) overwhelmingly dominates, followed by casual (0.695), temp (0.405), atemp (0.401), hr (0.394), hum (-0.323), weathersit (-0.142), and season (0.178). Registered users comprise ~80% of total demand.
  - **Key Temporal Patterns:** Two distinct daily peaks—8:00 AM (morning commute) and 5-6 PM (evening commute); strong registered user concentration during working hours; consistent low demand overnight (12 AM-5 AM) across all seasons.
  - **Seasonal Variation:** Summer shows highest peaks, winter the lowest; all seasons display same commute pattern, but with different intensity levels.
  - **Weather Impact:** Warmer temperature ( $r=0.405$ ) and lower humidity ( $r=-0.323$ ) correlate with higher demand; adverse weather slightly reduces demand ( $r=-0.142$ ).
  - **Conclusion:** Our preliminary expectations were confirmed and refined—registered users, time of day, and temperature are primary drivers; humidity and weather have weaker but meaningful negative associations.
- RQ2: Can clustering reveal distinct bike-usage regimes (e.g., commute, leisure, low-demand periods)?
- Before (expectations):
  - We expected at least low and high demand regimes to be revealed.
- After (our analysis):

- **K-Means Clustering (k=4, silhouette=0.4661):** Four distinct hourly regimes identified: (1) **Highest-demand commute** (avg 514 rentals, 96% working-day, 91% registered)—morning/evening peaks; (2) **High-demand mixed** (442 rentals, 12% working-day, 64% registered)—leisure/weekend focused; (3) **Medium-demand** (175 rentals, mixed usage); (4) **Low-demand off-peak** (51 rentals, late-night/early-morning).
- **DBSCAN Clustering (2 regimes + 5.1% noise):** Detected two dense usage regimes —(1) **High-demand regime** (peak evening commute, hr≈17.5, 823 avg rentals, silhouette=0.5335); (2) **Medium-high demand regime** (broader daytime, hr≈11.3, 166 avg rentals); ~5% sparse outliers.
- **Seasonality Clustering (2 regimes, silhouette=0.4252):** (1) **High-demand regime** (warm/dry: temp≈0.63, hum≈0.56, 329 avg rentals); (2) **Low-demand regime** (cool/humid: temp≈0.41, hum≈0.67, 98 avg rentals).
- **Conclusion:** Yes, clustering successfully revealed clear usage regimes. Primary distinction is commute-driven (working hours with registered users) vs. leisure/off-peak. Weather-based clustering confirms temperature and humidity strongly modulate demand. Regimes are interpretable and stable across algorithms.
- RQ 3: Can anomaly detection methods identify periods of unexpectedly high or low bike rental demand that may correspond to external events or unusual conditions not explicitly captured in the dataset?
- Before (expectations):
- We expected only a small fraction of hourly records to look anomalous.
- After
- **Anomaly detection identified a small but meaningful set of unusual periods.** All three selected methods flagged 965 rows ( 5.55% of the dataset).
- **Statistical outliers:** mainly detected globally extreme demand combinations, especially hours with unusually high cnt / registered values.
- **Isolation Forest:** found a similar number of anomalies and had moderate agreement with the statistical baseline ( Jaccard = 0.537 ), suggesting both methods capture broadly similar global anomalies.
- **LOF:** also flagged 965 rows, but with very low overlap with the other methods ( 0.106 with Statistical, 0.142 with Isolation Forest), meaning it captures more **local/contextual anomalies** rather than only globally extreme points.
- **Overall:** Anomaly detection successfully identified unusual periods. These are best interpreted as candidate anomalous demand hours.
- RQ 4: Is there noise in the data? If so, what can be done with it?
- Before (expectations):
- We expected little data quality noise because the dataset is structured and well prepared, without NaN values, but we expected some sparse or irregular observations at the edges of normal demand regimes.
- After
- **DBSCAN identified 879 rows ( 5.06% ) as noise**, meaning these points do not belong to the main dense behavioral regimes in the chosen feature space. The question is, are they related with outliers or not?
- The noise is considered according to us as non-pattern data rather than error.
- Some of these points may correspond to very low-demand hours, unusual combinations of demand and weather, or rare edge cases between normal regimes.
- **What could we done with it?**
- keep it and analyze it separately as irregular behavior
- add a noise flag as an informative feature -> to filter it later

- remove it
- RQ 5: How to decide which outliers are valid. And how to compare them with the noise from clustering?
- Before (expectations):
  - We expected that not every detected outlier would be equally trustworthy.
  - We also expected only partial overlap between anomaly-detection outliers and DBSCAN noise, because they represent different concepts.
- After:
  - The rows found by both **Statistical Outlier** and **Isolation Forest** are stronger candidates for globally unusual demand periods.
  - **LOF outliers** are also valid candidates, but they should be interpreted differently: they are more likely **local anomalies**.
  - **Clustering noise is not the same as an outlier.**
  - **DBSCAN noise** -> not part of any dense cluster.
  - **Outlier detection** -> unusual according to a specific anomaly criterion

## 0. Reproducibility and Setup

---

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from ucimlrepo import fetch_ucirepo
from sklearn.cluster import KMeans
from sklearn.cluster import DBSCAN
from sklearn.neighbors import NearestNeighbors, LocalOutlierFactor
from matplotlib.colors import ListedColormap
from matplotlib.patches import Patch
# Clustering Evaluation Metrics - Required measures:
# 1. Contingency table, 2. Purity, 3. F-measure, 4. Maximum matching,
# 5. Conditional entropy, 6. Mutual information
from scipy.optimize import linear_sum_assignment
from scipy.stats import entropy as scipy_entropy
from sklearn.metrics import mutual_info_score, cluster, silhouette_score
# Visualize K-Means contingency tables with normalized values
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.lines import Line2D
from sklearn.ensemble import IsolationForest
```

# 1. Dataset Description and Loading

## 1.1 Dataset Overview

- Source: <https://archive.ics.uci.edu/dataset/275/bike+sharing+dataset>
- Number of instances: 17,379 hourly records
- Number of features / entities: 17 variables in the hourly table
- Missing values: 0 missing values in the loaded hourly table
- Basic statistics: summarized in the following output tables

```
In [2]: bike_sharing = fetch_ucirepo(id=275)

print(f"Dataset {bike_sharing.metadata['name']}")
print(f"Description: {bike_sharing.metadata['abstract']}")
print(f"Created {bike_sharing.metadata['year_of_dataset_creation']}")
# data
df = bike_sharing.data.original
df.head()
```

Dataset Bike Sharing

Description: This dataset contains the hourly and daily count of rental bikes between years 2011 and 2012 in Capital bikeshare system with the corresponding weather and seasonal information.  
Created 2013

Out[2]:

|   | instant | dteday     | season | yr | mnth | hr | holiday | weekday |
|---|---------|------------|--------|----|------|----|---------|---------|
| 0 | 1       | 2011-01-01 | 1      | 0  | 1    | 0  | 0       | 6       |
| 1 | 2       | 2011-01-01 | 1      | 0  | 1    | 1  | 0       | 6       |
| 2 | 3       | 2011-01-01 | 1      | 0  | 1    | 2  | 0       | 6       |
| 3 | 4       | 2011-01-01 | 1      | 0  | 1    | 3  | 0       | 6       |
| 4 | 5       | 2011-01-01 | 1      | 0  | 1    | 4  | 0       | 6       |

```
In [3]: print(f"Dataset shape: {df.shape}")
        print(f"Dataset columns: {df.columns}")
        print(f"missing values: {df.isnull().sum()}")
```

```
Dataset shape: (17379, 17)
Dataset columns: Index(['instant', 'dteday', 'season', 'yr',
                        'mnth', 'hr', 'holiday', 'weekday',
                        'workingday', 'weathersit', 'temp', 'atemp', 'hum',
                        'windspeed',
                        'casual', 'registered', 'cnt'],
                        dtype='object')
missing values: instant      0
dteday      0
season      0
yr          0
mnth        0
hr          0
holiday     0
weekday     0
workingday  0
weathersit   0
temp        0
atemp       0
hum         0
windspeed   0
casual      0
registered  0
cnt         0
dtype: int64
```

There are 17 columns:

- **instant:** record index
- **dteday:** date
- **season:**
  - 1:winter
  - 2:spring
  - 3:summer
  - 4:fall
- **yr:** year (0: 2011, 1: 2012)
- **mnth:** month (1 to 12)
- **hr:** hour (0 to 23)
- **holiday:** whether the day is a holiday (from the Washington, D.C. holiday calendar)
- **weekday:** day of the week
- **workingday:** 1 if the day is neither weekend nor holiday, otherwise 0
- **weathersit:**
  - 1: Clear, Few clouds, Partly cloudy
  - 2: Mist + Cloudy, Mist + Broken clouds, Mist + Few clouds, Mist
  - 3: Light Snow, Light Rain + Thunderstorm + Scattered clouds, Light Rain + Scattered clouds
  - 4: Heavy Rain + Ice Pellets + Thunderstorm + Mist, Snow + Fog
- **temp:** normalized temperature in Celsius, scaled from  $t_{\min} = -8$  to  $t_{\max} = +39$  (hourly)
- **atemp:** normalized feeling temperature in Celsius, scaled from  $t_{\min} = -16$  to  $t_{\max} = +50$  (hourly)
- **hum:** normalized humidity (divided by 100)
- **windspeed:** normalized wind speed (divided by 67)
- **casual:** count of casual users
- **registered:** count of registered users
- **cnt:** total count of rented bikes (casual + registered)

## 1.2 Exploratory Data Analysis

- Basic statistics
- Distributions
- Sparsity / density

This section follows the lecture focus on exploratory statistics, variable distributions, and identifying dense vs. sparse regions in feature space.

### 1.2.1 Basic Statistics (from the Lecture 1)

```
In [4]: numeric_cols = df.select_dtypes(include=np.number).columns

# T here means transpose, which flips the rows and columns of the
DataFrame
basic_stats_extended = df[numeric_cols].describe().T
# replace 50% with median, to make it readable
basic_stats_extended["median"] = df[numeric_cols].median()
basic_stats_extended["mode"] = df[numeric_cols].mode().iloc[0]
basic_stats_extended["iqr"] = basic_stats_extended["75%"] -
basic_stats_extended["25%"]
basic_stats_extended["skew"] = df[numeric_cols].skew()

basic_stats_extended = basic_stats_extended[
    ["count", "mean", "std", "min", "25%", "median", "mode", "75%",
    "max", "iqr", "skew"]
].round(3)

print("Extended basic statistics (numeric features):")
basic_stats_extended
```

Extended basic statistics (numeric features):

```
Out[4]:
```

|            | count   | mean     | std      | min  | 25%      | median   |
|------------|---------|----------|----------|------|----------|----------|
| instant    | 17379.0 | 8690.000 | 5017.029 | 1.00 | 4345.500 | 8690.000 |
| season     | 17379.0 | 2.502    | 1.107    | 1.00 | 2.000    | 3.000    |
| yr         | 17379.0 | 0.503    | 0.500    | 0.00 | 0.000    | 1.000    |
| mnth       | 17379.0 | 6.538    | 3.439    | 1.00 | 4.000    | 7.000    |
| hr         | 17379.0 | 11.547   | 6.914    | 0.00 | 6.000    | 12.000   |
| holiday    | 17379.0 | 0.029    | 0.167    | 0.00 | 0.000    | 0.000    |
| weekday    | 17379.0 | 3.004    | 2.006    | 0.00 | 1.000    | 3.000    |
| workingday | 17379.0 | 0.683    | 0.465    | 0.00 | 0.000    | 1.000    |

|            | count   | mean    | std     | min  | 25%    | median  |
|------------|---------|---------|---------|------|--------|---------|
| weathersit | 17379.0 | 1.425   | 0.639   | 1.00 | 1.000  | 1.000   |
| temp       | 17379.0 | 0.497   | 0.193   | 0.02 | 0.340  | 0.500   |
| atemp      | 17379.0 | 0.476   | 0.172   | 0.00 | 0.333  | 0.485   |
| hum        | 17379.0 | 0.627   | 0.193   | 0.00 | 0.480  | 0.630   |
| windspeed  | 17379.0 | 0.190   | 0.122   | 0.00 | 0.104  | 0.194   |
| casual     | 17379.0 | 35.676  | 49.305  | 0.00 | 4.000  | 17.000  |
| registered | 17379.0 | 153.787 | 151.357 | 0.00 | 34.000 | 115.000 |
| cnt        | 17379.0 | 189.463 | 181.388 | 1.00 | 40.000 | 142.000 |

Where:

- **count** - number of non-null values in each column (here, 17,379 for the hourly table)
- **mean** - average value in each column
- **std** - standard deviation (how spread out the values are around the mean)
- **min** - smallest value in each column
- **25%** - value at the first quartile
- **median (50%)** - median value
- **mode** - the most frequent value (for ties, the first mode is shown)
- **75%** - value at the third quartile
- **max** - largest value in each column
- **iqr (75% - 25%)** - a robust spread measure, interquartile range
- **skew** - shape/asymmetry (positive = right tail, negative = left tail, near 0 = more symmetric)



#### Observations:

- **mean vs median** - are almost symmetric

According to rows:

- **instant**: row index, so nothing interesting here
- **season (1..4)**: **mean** 2.502 (looks like seasons are balanced), **skew** near 0
- **yr (0=2011, 1=2012)**: **mean** 0.503 (roughly 50/50 split between years)
- **mnth (1..12)**: quartiles (**25%, 50%, 75%**) 4, 7, 10, **skew** near-zero, month coverage looks balanced
- **hr (0..23)**: quartiles (**25%, 50%, 75%**) 6, 12, 18, **skew** near-zero, full-day coverage looks balanced
- **holiday (0/1)**: **mean** 0.029 means only ~2.9% records are holidays, **median**=0, **75%**=0, **iqr**=0, **skew**=5.639 (mostly non-holiday).
- **weekday (0..6)**: **skew** near-zero, week coverage looks balanced
- **workingday (0/1)**: **mean** 0.683 means ~68.3% working-day hours, **skew** -0.785 which supports mean theory
- **weathersit (1..4)**: **mean** 1.425, **median** 1, **75%**=2, **skew** 1.228, severe weather is rare
- **temp (normalized 0..1)**: **mean** 0.497, **median** 0.500, **skew** -0.006 -> near-symmetric
- **atemp (normalized)**: **skew** slightly left-skewed (-0.090)
- **hum (normalized)**: **mean** 0.627, **median** 0.630
- **windspeed (normalized)**: **mean** 0.190, **median** 0.194, **skew** 0.575, mostly moderate wind with some high-wind tail
- **casual**: **mean** 35.676, **median** 17, **skew** 2.499 -> very right-skewed
- **registered**: **mean** 153.787, **median** 115, **skew** 1.558 0> right-skewed
- **cnt**: overall pattern is right-skewed

## Covariance / Correlation

- summarization of pairwise covariance and correlation among numeric variables (matrices)

Covariance:

- **positive covariance**: when values (X, Y) is above its mean
- **negative covariance**: when X is above mean, Y tends to be below mean
- **near 0**: two variables do not show a clear straight-line relationship

Correlation:

- covariance normalized by both standard deviations (between -1 and +1)

```

In [5]: cov_matrix = df[numeric_cols].cov()
        corr_matrix = df[numeric_cols].corr()

        print("Covariance matrix (numeric):")
        display(cov_matrix.round(3))

        print("Correlation matrix (numeric):")
        display(corr_matrix.round(3))

        # remove corr with itself, and sort by absolute value
        corr_with_cnt = corr_matrix["cnt"].drop("cnt")
        top_abs_with_cnt_corr =
        corr_with_cnt.reindex(corr_with_cnt.abs().sort_values(ascending=False).i
        ndex)
        print("Strongest correlations with cnt (absolute ranking):")
        display(top_abs_with_cnt_corr.head(10).round(3))

```

Covariance matrix (numeric):

|            | instant       | season   | yr       | mnth     | hr       |
|------------|---------------|----------|----------|----------|----------|
| instant    | 2.517058e+07  | 2243.844 | 2172.443 | 8439.269 | -165.637 |
| season     | 2.243844e+03  | 1.225    | -0.006   | 3.161    | -0.047   |
| yr         | 2.172443e+03  | -0.006   | 0.250    | -0.018   | -0.013   |
| mnth       | 8.439269e+03  | 3.161    | -0.018   | 11.825   | -0.137   |
| hr         | -1.656370e+02 | -0.047   | -0.013   | -0.137   | 47.809   |
| holiday    | 1.234800e+01  | -0.002   | 0.001    | 0.011    | 0.001    |
| weekday    | 1.365400e+01  | -0.005   | -0.004   | 0.072    | -0.049   |
| workingday | -7.976000e+00 | 0.007    | -0.001   | -0.006   | 0.007    |
| weathersit | -4.554100e+01 | -0.010   | -0.006   | 0.012    | -0.089   |
| temp       | 1.315560e+02  | 0.067    | 0.004    | 0.134    | 0.183    |
| atemp      | 1.186480e+02  | 0.061    | 0.003    | 0.123    | 0.159    |
| hum        | 9.270000e+00  | 0.032    | -0.008   | 0.109    | -0.369   |
| windspeed  | -4.573000e+01 | -0.020   | -0.001   | -0.057   | 0.116    |
| casual     | 3.915671e+04  | 6.560    | 3.520    | 11.607   | 102.684  |
| registered | 2.141754e+05  | 29.190   | 19.199   | 63.641   | 391.555  |
| cnt        | 2.533321e+05  | 35.750   | 22.719   | 75.248   | 494.239  |

Correlation matrix (numeric):

|            | instant | season | yr     | mnth   | hr     | holiday | wee   |
|------------|---------|--------|--------|--------|--------|---------|-------|
| instant    | 1.000   | 0.404  | 0.866  | 0.489  | -0.005 | 0.015   | 0.00  |
| season     | 0.404   | 1.000  | -0.011 | 0.830  | -0.006 | -0.010  | -0.00 |
| yr         | 0.866   | -0.011 | 1.000  | -0.010 | -0.004 | 0.007   | -0.00 |
| mnth       | 0.489   | 0.830  | -0.010 | 1.000  | -0.006 | 0.018   | 0.00  |
| hr         | -0.005  | -0.006 | -0.004 | -0.006 | 1.000  | 0.000   | -0.00 |
| holiday    | 0.015   | -0.010 | 0.007  | 0.018  | 0.000  | 1.000   | -0.00 |
| weekday    | 0.001   | -0.002 | -0.004 | 0.010  | -0.003 | -0.102  | 1.00  |
| workingday | -0.003  | 0.014  | -0.002 | -0.003 | 0.002  | -0.252  | 0.00  |
| weathersit | -0.014  | -0.015 | -0.019 | 0.005  | -0.020 | -0.017  | 0.00  |
| temp       | 0.136   | 0.312  | 0.041  | 0.202  | 0.138  | -0.027  | -0.00 |
| atemp      | 0.138   | 0.319  | 0.039  | 0.208  | 0.134  | -0.031  | -0.00 |
| hum        | 0.010   | 0.151  | -0.084 | 0.164  | -0.276 | -0.011  | -0.00 |
| windspeed  | -0.075  | -0.150 | -0.009 | -0.135 | 0.137  | 0.004   | 0.00  |
| casual     | 0.158   | 0.120  | 0.143  | 0.068  | 0.301  | 0.032   | 0.00  |
| registered | 0.282   | 0.174  | 0.254  | 0.122  | 0.374  | -0.047  | 0.00  |
| cnt        | 0.278   | 0.178  | 0.250  | 0.121  | 0.394  | -0.031  | 0.00  |

Strongest correlations with cnt (absolute ranking):

```

registered    0.972
casual        0.695
temp          0.405
atemp         0.401
hr            0.394
hum           -0.323
instant       0.278
yr            0.250
season        0.178
weathersit    -0.142
Name: cnt, dtype: float64

```

#### Observations:

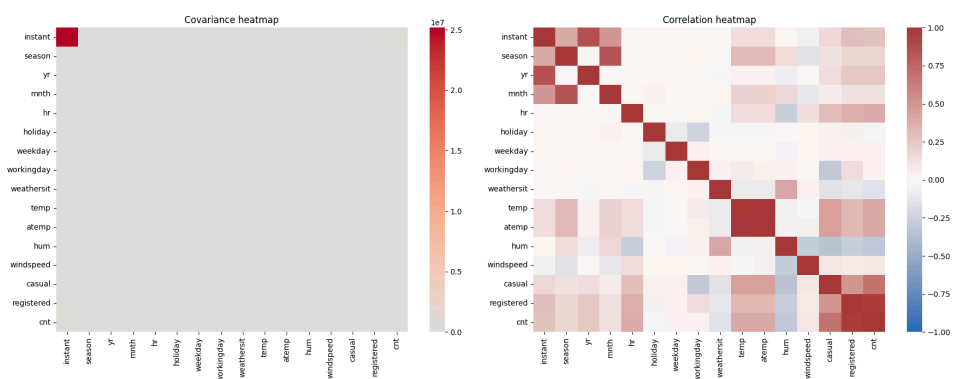
- **registered:** strong relationship, expected cause it is a part of *cnt*
- **casual:** strong relationship, expected cause it is a part of *cnt*
- **temp:** positive relationship (warmer conditions might be associated with more rentals)
- **atemp:** positive relationship (warmer conditions might be associated with more rentals)
- **hr:** positive relationship (hour matters, but correlation only captures part of it)
- **hum:** negative relationship (higher humidity tends to be associated with fewer rentals)
- **instant:** index, so useless
- **yr:** weaker relationship (2012 has slightly more impact than 2011)
- **season:** weaker relationship (it is more categorical, so hard to say here)
- **weathersit:** weaker negative relationship (worse weather categories are linked with slightly lower interest)

```
In [6]: fig, axes = plt.subplots(1, 2, figsize=(18, 7))

# center=0 -> color is centered at zero (positive vs negative shown
clearly)
sns.heatmap(cov_matrix, cmap="coolwarm", center=0, ax=axes[0])
axes[0].set_title("Covariance heatmap")

# vmin=-1, vmax=1 -> fix the color scale to full correlation range
sns.heatmap(corr_matrix, cmap="vlag", center=0, vmin=-1, vmax=1,
ax=axes[1])
axes[1].set_title("Correlation heatmap")

plt.tight_layout()
plt.show()
```



#### Observations:

- **Covariance Heatmap:**

- mostly grey, might be because of *instant* feature, which has a very large scale
- not useful plot for us

- **Correlation heatmap:**

- since data are normalised, it's more useful than *Covariance Heatmap*
- dark red on diagonal -> each variable with itself (1.0)
- *temp* and *atemp* are strongly positively correlated (near duplicate information), which makes sense, since they are very similar metrics
- *cnt* is strongly positive with *registered* and *casual*, as expected
- *cnt* is positive with *temp* and *hr*, this confirms our observation from the previous matrices section
- *cnt* is negative with *hum* and slightly negative with worse *weathersit*

## 1.2.2 Distributions

We inspect numerical and categorical distributions because clustering and outlier behavior depend on where probability mass is concentrated.

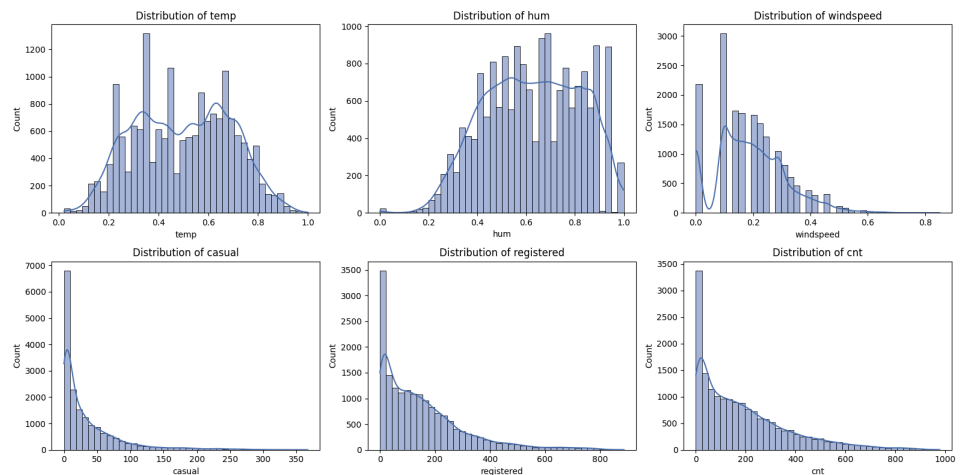
### Count Distribution (numerical)

- how many times the value in x-axis appears in the y-axis

```
In [7]: dist_cols = ["temp", "hum", "windspeed", "casual", "registered", "cnt"]

fig, axes = plt.subplots(2, 3, figsize=(16, 8))
for ax, col in zip(axes.flatten(), dist_cols):
    sns.histplot(df[col], bins=40, kde=True, ax=ax, color="#4C72B0")
    ax.set_title(f"Distribution of {col}")
    ax.set_xlabel(col)

# makes plots readable and displays them
plt.tight_layout()
plt.show()
```



#### #### Observations:

- **temp:** not perfectly bell-shaped -> mixture of models (more peaks), could be because of seasons
- **hum:** most values are in the range 0.4–0.9 (medium-high humidity), low humidity is more rare
- **windspeed:** long tail to high wind values, high wind conditions are uncommon?
- **casual:** low casual rentals, only few have very high casual rentals
- **registered:** similarly to **casual**, but better
- **cnt:** shape close to **registered**

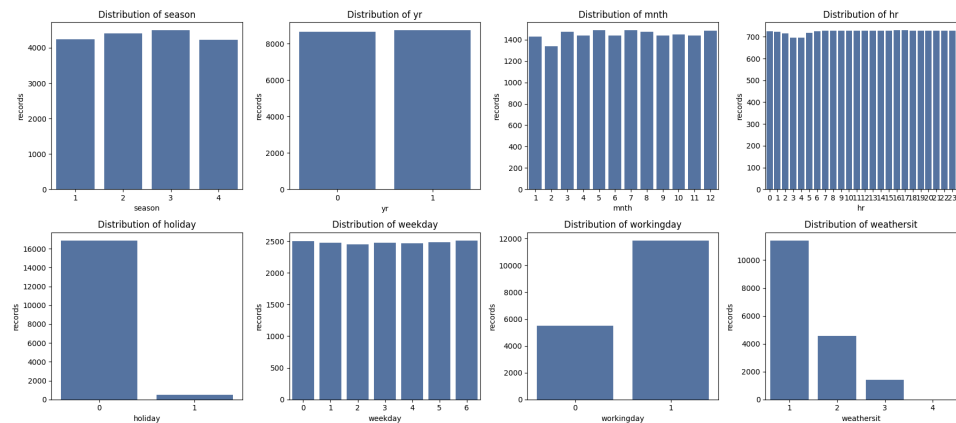
## Count Distribution (categorical)

- how many times the value in x-axis appears in the y-axis

```
In [8]: category_cols = ["season", "yr", "mnth", "hr", "holiday", "weekday",
                        "workingday", "weathersit"]

fig, axes = plt.subplots(2, 4, figsize=(18, 8))
for ax, col in zip(axes.flatten(), category_cols):
    counts = df[col].value_counts().sort_index()
    sns.barplot(x=counts.index, y=counts.values, ax=ax, color="#4C72B0")
    ax.set_title(f"Distribution of {col}")
    ax.set_xlabel(col)
    ax.set_ylabel("records")

plt.tight_layout()
plt.show()
```



#### Observations:

- **season:** balanced, 3 slightly higher (summer)
- **yr:** balanced
- **mnth:** coverage is balanced
- **hr:** surprisingly balanced
- **holiday:** imbalanced, holidays are a small minority
- **weekday:** balanced
- **workingday:** imbalanced, weekdays\holidays are fewer
- **weathersit:** imbalanced, the weather is mostly good

### 1.2.3 Sparsity/Density

- **Sparsity:** many values are exactly 0 (or missing) in a feature
- **Density:** how concentrated records are in regions of feature space (many points close together)

In EDA we used only descriptive code, no clustering yet

## Sparsity

```
In [9]: sparsity = pd.DataFrame({
        # nan
        "missing_fraction": df[numeric_cols].isna().mean(),
        # exactly zero
        "zero_fraction": (df[numeric_cols] == 0).mean(),
        # non-zero values
        "non_zero_fraction": (df[numeric_cols] != 0).mean()
        # puts most sparse by zeros features on top
    }).sort_values("zero_fraction", ascending=False).round(3)

    print("Sparsity summary (numeric features):")
    display(sparsity)

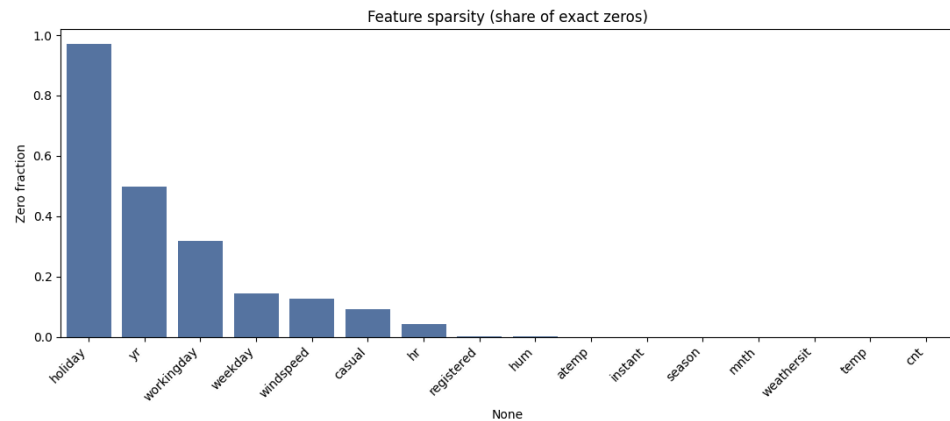
    plt.figure(figsize=(11, 5))
    sns.barplot(x=sparsity.index, y=sparsity["zero_fraction"],
               color="#4C72B0")
    plt.xticks(rotation=45, ha="right")
    plt.ylabel("Zero fraction")
    plt.title("Feature sparsity (share of exact zeros)")
    plt.tight_layout()
    plt.show()
```

Sparsity summary (numeric features):

|            | missing_fraction | zero_fraction | non_zero_fraction |
|------------|------------------|---------------|-------------------|
| holiday    | 0.0              | 0.971         | 0.029             |
| yr         | 0.0              | 0.497         | 0.503             |
| workingday | 0.0              | 0.317         | 0.683             |
| weekday    | 0.0              | 0.144         | 0.856             |
| windspeed  | 0.0              | 0.125         | 0.875             |
| casual     | 0.0              | 0.091         | 0.909             |
| hr         | 0.0              | 0.042         | 0.958             |
| registered | 0.0              | 0.001         | 0.999             |
| hum        | 0.0              | 0.001         | 0.999             |
| atemp      | 0.0              | 0.000         | 1.000             |
| instant    | 0.0              | 0.000         | 1.000             |
| season     | 0.0              | 0.000         | 1.000             |
| mnth       | 0.0              | 0.000         | 1.000             |
| weathersit | 0.0              | 0.000         | 1.000             |



|      | missing_fraction | zero_fraction | non_zero_fraction |
|------|------------------|---------------|-------------------|
| temp | 0.0              | 0.000         | 1.000             |
| cnt  | 0.0              | 0.000         | 1.000             |



#### #### Observations:

- higher bar = more zeros = more sparse feature
- **holiday**: means 97.1% of rows are non-holidays, only 2.9% are holidays
- **yr**: means about half the rows are year 2011 and half year 2012 (roughly balanced)
- **workingday**: means 31.7% non-working days, 68.3% working days
- **weekday**: does not mean sparse in the usual way, it means category 0 appears 14.4% of the time
- **hr**: similarly to *weekday*
- **registered** and **hum**: near 0.001 mean almost never zero
- as we mentioned earlier, no missing values

## Density

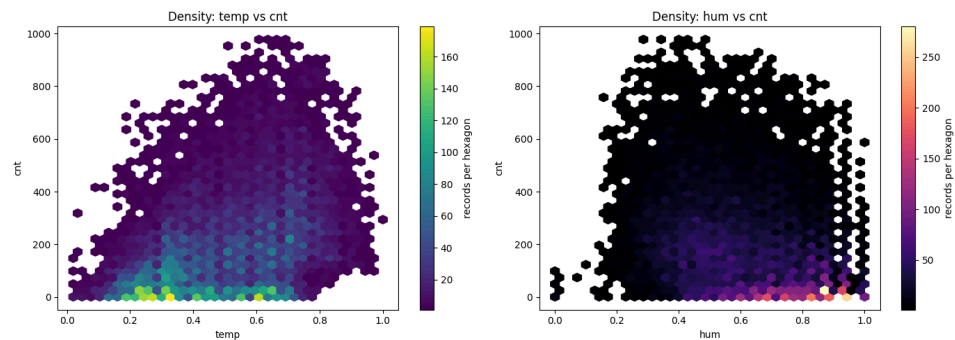
- let's see comparison of **temp**, **hum** and **cnt** since we saw correlation between them

```
In [10]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))

hexagon_bin_1 = axes[0].hexbin(df["temp"], df["cnt"], gridsize=35,
                                cmap="viridis", mincnt=1)
axes[0].set_title("Density: temp vs cnt")
axes[0].set_xlabel("temp")
axes[0].set_ylabel("cnt")
fig.colorbar(hexagon_bin_1, ax=axes[0], label="records per hexagon")

hexagon_bin_2 = axes[1].hexbin(df["hum"], df["cnt"], gridsize=35,
                                cmap="magma", mincnt=1)
axes[1].set_title("Density: hum vs cnt")
axes[1].set_xlabel("hum")
axes[1].set_ylabel("cnt")
fig.colorbar(hexagon_bin_2, ax=axes[1], label="records per hexagon")

plt.tight_layout()
plt.show()
```



#### Observations:

- **temp vs cnt:**

- as **temp** moves from cold to moderate, higher **cnt** becomes more possible
- so warmer temperature = more rentals

- **hum vs cnt:**

- high humidity (**hum** near 0.8–1.0) is mostly associated with low **cnt** (similarly with high temperature)
- higher **cnt** values are more common from low to medium humidity (**hum**)
- so less humidity = more rentals

## 1.3 Cleaning & Transformations

- Handling missing values
- Feature engineering
- Graph construction (if Checkpoint 2)

### 1.3.1 Handling missing values

The bike-sharing hourly data has no missing values.

### 1.3.2 Feature engineering

Drop non-informative columns:

- **instant**: sequential index with no analytical signal, as we also saw in EDA

Extract day from dteday and rename column:

- **dteday**: redundant information, as 'yr' and 'mnth' exist as separate columns, but we need **day** as part of the day during our observation

```
In [11]: df_clean = df.drop(columns=["instant"])

df_clean["dteday"] = pd.to_datetime(df_clean["dteday"]).dt.day
df_clean = df_clean.rename(columns={"dteday": "day"})

print(f"Columns after cleaning: {df_clean.columns.tolist()}")
print(f"Shape: {df_clean.shape}")
```

```
Columns after cleaning: ['day', 'season', 'yr', 'mnth', 'hr',
'holiday', 'weekday', 'workingday', 'weathersit', 'temp', 'atemp',
'hum', 'windspeed', 'casual', 'registered', 'cnt']
Shape: (17379, 16)
```

Validate data integrity for data consistency issues: casual users + registered users should equal total count

```
In [12]: validation_error = (df_clean["casual"] + df_clean["registered"] -
df_clean["cnt"]).abs().sum()

print(f"Data integrity check (should be 0): {validation_error}")

if validation_error == 0:
    print("Data are consistent: casual + registered = cnt")
else:
    print("Data are inconsistent!")
```

```
Data integrity check (should be 0): 0
Data are consistent: casual + registered = cnt
```

### 1.3.3 Feature engineering: categorical encoding

```
In [13]: season_labels = {1: "winter", 2: "spring", 3: "summer", 4: "fall"}
year_labels = {0: "2011", 1: "2012"}
weekday_labels = {0: "Sunday", 1: "Monday", 2: "Tuesday", 3:
"Wednesday", 4: "Thursday", 5: "Friday", 6: "Saturday"}
weathersit_labels = {1: "Clear/Few clouds", 2: "Mist/Cloudy", 3: "Light
Snow/Rain", 4: "Heavy Rain/Snow"}

season_dummies = pd.get_dummies(df_clean["season"].map(season_labels),
prefix="season")
year_dummies = pd.get_dummies(df_clean["yr"].map(year_labels),
prefix="year")
weekday_dummies =
pd.get_dummies(df_clean["weekday"].map(weekday_labels),
prefix="weekday")
weathersit_dummies =
pd.get_dummies(df_clean["weathersit"].map(weathersit_labels),
prefix="weathersit")

df_encoded = pd.concat([
    df_clean.drop(columns=["season", "weathersit", "weekday", "yr"]),
    season_dummies,
    year_dummies,
    weekday_dummies,
    weathersit_dummies
], axis=1)

print(f"Columns after encoding: {df_encoded.columns.tolist()}")
print(f"Shape after encoding: {df_encoded.shape}")
print(f"\nData types:\n{df_encoded.dtypes}")
```

```
Columns after encoding: ['day', 'mnth', 'hr', 'holiday',
'workingday', 'temp', 'atemp', 'hum', 'windspeed', 'casual',
'registered', 'cnt', 'season_fall', 'season_spring',
'season_summer', 'season_winter', 'year_2011', 'year_2012',
'weekday_Friday', 'weekday_Monday', 'weekday_Saturday',
'weekday_Sunday', 'weekday_Thursday', 'weekday_Tuesday',
'weekday_Wednesday', 'weathersit_Clear/Few clouds',
'weathersit_Heavy Rain/Snow', 'weathersit_Light Snow/Rain',
'weathersit_Mist/Cloudy']
Shape after encoding: (17379, 29)
```

Data types:

|            |         |
|------------|---------|
| day        | int32   |
| mnth       | int64   |
| hr         | int64   |
| holiday    | int64   |
| workingday | int64   |
| temp       | float64 |
| atemp      | float64 |
| hum        | float64 |
| windspeed  | float64 |

|                             |        |
|-----------------------------|--------|
| casual                      | int64  |
| registered                  | int64  |
| cnt                         | int64  |
| season_fall                 | bool   |
| season_spring               | bool   |
| season_summer               | bool   |
| season_winter               | bool   |
| year_2011                   | bool   |
| year_2012                   | bool   |
| weekday_Friday              | bool   |
| weekday_Monday              | bool   |
| weekday_Saturday            | bool   |
| weekday_Sunday              | bool   |
| weekday_Thursday            | bool   |
| weekday_Tuesday             | bool   |
| weekday_Wednesday           | bool   |
| weathersit_Clear/Few clouds | bool   |
| weathersit_Heavy Rain/Snow  | bool   |
| weathersit_Light Snow/Rain  | bool   |
| weathersit_Mist/Cloudy      | bool   |
| dtype:                      | object |

## Feature Scaling

---

## 2. Module 1 — Vector-Space Analysis

---

Standardize numerical features to zero mean and unit variance to make scales comparable across variables.

- This reduces dominance of large-scale variables in distance-based methods.
- The `hr` variable is kept unscaled in this version because time-of-day can be handled with cyclical encoding ( `sin` / `cos` ) in later steps.

```
In [14]: scaler = StandardScaler()

numerical_cols = [
    "temp", "atemp", "hum", "windspeed",
    "cnt", "casual", "registered"
]

df_encoded[numerical_cols] = scaler.fit_transform(
    df_encoded[numerical_cols]
)

print("Scaling verification (should be ~0 mean, ~1 std):")
print(f"\nMeans:\n{df_encoded[numerical_cols].mean().round(4)}")
print(f"\nStd Devs:\n{df_encoded[numerical_cols].std().round(4)}")
print(f"\nNote: 'hr' column kept unscaled (range 0-23)")
```

Scaling verification (should be ~0 mean, ~1 std):

Means:

|            |      |
|------------|------|
| temp       | 0.0  |
| atemp      | -0.0 |
| hum        | 0.0  |
| windspeed  | 0.0  |
| cnt        | -0.0 |
| casual     | 0.0  |
| registered | -0.0 |

dtype: float64

Std Devs:

|            |     |
|------------|-----|
| temp       | 1.0 |
| atemp      | 1.0 |
| hum        | 1.0 |
| windspeed  | 1.0 |
| cnt        | 1.0 |
| casual     | 1.0 |
| registered | 1.0 |

dtype: float64

Note: 'hr' column kept unscaled (range 0-23)

## Final Dataset Check

```
In [15]: # Final cleaned and processed dataset
print("Dataset Overview:")
print(f"Shape: {df_encoded.shape}")
print(f"\nFirst 5 rows:")
print(df_encoded.head())
print(f"\nData Info:")
df_encoded.info()
print(f"\nBasic Statistics:")
print(df_encoded.describe().round(3))
```

Dataset Overview:

Shape: (17379, 29)

First 5 rows:

|          | day | mnth | hr | holiday | workingday | temp      | atemp     |
|----------|-----|------|----|---------|------------|-----------|-----------|
| 0        | 1   | 1    | 0  | 0       | 0          | -1.334648 | -1.093281 |
| 0.947372 |     |      |    |         |            |           |           |
| 1        | 1   | 1    | 1  | 0       | 0          | -1.438516 | -1.181732 |
| 0.895539 |     |      |    |         |            |           |           |
| 2        | 1   | 1    | 2  | 0       | 0          | -1.438516 | -1.181732 |
| 0.895539 |     |      |    |         |            |           |           |
| 3        | 1   | 1    | 3  | 0       | 0          | -1.334648 | -1.093281 |
| 0.636370 |     |      |    |         |            |           |           |
| 4        | 1   | 1    | 4  | 0       | 0          | -1.334648 | -1.093281 |
| 0.636370 |     |      |    |         |            |           |           |

|       | windspeed | casual    | ... | weekday_Monday | weekday_Saturday |
|-------|-----------|-----------|-----|----------------|------------------|
| 0     | -1.553889 | -0.662755 | ... | False          | True             |
| False |           |           |     |                |                  |
| 1     | -1.553889 | -0.561343 | ... | False          | True             |
| False |           |           |     |                |                  |
| 2     | -1.553889 | -0.622190 | ... | False          | True             |
| False |           |           |     |                |                  |
| 3     | -1.553889 | -0.662755 | ... | False          | True             |
| False |           |           |     |                |                  |
| 4     | -1.553889 | -0.723603 | ... | False          | True             |
| False |           |           |     |                |                  |

|   | weekday_Thursday | weekday_Tuesday | weekday_Wednesday | \ |
|---|------------------|-----------------|-------------------|---|
| 0 | False            | False           | False             |   |
| 1 | False            | False           | False             |   |
| 2 | False            | False           | False             |   |
| 3 | False            | False           | False             |   |
| 4 | False            | False           | False             |   |

|   | weathersit_Clear/Few clouds | weathersit_Heavy Rain/Snow | \ |
|---|-----------------------------|----------------------------|---|
| 0 | True                        | False                      |   |
| 1 | True                        | False                      |   |
| 2 | True                        | False                      |   |

|   |      |       |
|---|------|-------|
| 3 | True | False |
| 4 | True | False |

|   |                            |                        |
|---|----------------------------|------------------------|
|   | weathersit_Light Snow/Rain | weathersit_Mist/Cloudy |
| 0 | False                      | False                  |
| 1 | False                      | False                  |
| 2 | False                      | False                  |
| 3 | False                      | False                  |
| 4 | False                      | False                  |

[5 rows x 29 columns]

Data Info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 17379 entries, 0 to 17378

Data columns (total 29 columns):

| #   | Column                      | Non-Null Count | Dtype   |
|-----|-----------------------------|----------------|---------|
| --- | -----                       | -----          | -----   |
| 0   | day                         | 17379 non-null | int32   |
| 1   | mnth                        | 17379 non-null | int64   |
| 2   | hr                          | 17379 non-null | int64   |
| 3   | holiday                     | 17379 non-null | int64   |
| 4   | workingday                  | 17379 non-null | int64   |
| 5   | temp                        | 17379 non-null | float64 |
| 6   | atemp                       | 17379 non-null | float64 |
| 7   | hum                         | 17379 non-null | float64 |
| 8   | windspeed                   | 17379 non-null | float64 |
| 9   | casual                      | 17379 non-null | float64 |
| 10  | registered                  | 17379 non-null | float64 |
| 11  | cnt                         | 17379 non-null | float64 |
| 12  | season_fall                 | 17379 non-null | bool    |
| 13  | season_spring               | 17379 non-null | bool    |
| 14  | season_summer               | 17379 non-null | bool    |
| 15  | season_winter               | 17379 non-null | bool    |
| 16  | year_2011                   | 17379 non-null | bool    |
| 17  | year_2012                   | 17379 non-null | bool    |
| 18  | weekday_Friday              | 17379 non-null | bool    |
| 19  | weekday_Monday              | 17379 non-null | bool    |
| 20  | weekday_Saturday            | 17379 non-null | bool    |
| 21  | weekday_Sunday              | 17379 non-null | bool    |
| 22  | weekday_Thursday            | 17379 non-null | bool    |
| 23  | weekday_Tuesday             | 17379 non-null | bool    |
| 24  | weekday_Wednesday           | 17379 non-null | bool    |
| 25  | weathersit_Clear/Few clouds | 17379 non-null | bool    |
| 26  | weathersit_Heavy Rain/Snow  | 17379 non-null | bool    |
| 27  | weathersit_Light Snow/Rain  | 17379 non-null | bool    |
| 28  | weathersit_Mist/Cloudy      | 17379 non-null | bool    |

dtypes: bool(17), float64(7), int32(1), int64(4)

memory usage: 1.8 MB

Basic Statistics:

|        |           |           |           |           |            |
|--------|-----------|-----------|-----------|-----------|------------|
|        | day       | mnth      | hr        | holiday   | workingday |
| temp \ |           |           |           |           |            |
| count  | 17379.000 | 17379.000 | 17379.000 | 17379.000 | 17379.000  |
|        | 17379.000 |           |           |           |            |
| mean   | 15.683    | 6.538     | 11.547    | 0.029     | 0.683      |
|        | 0.000     |           |           |           |            |



|        |        |        |        |       |       |
|--------|--------|--------|--------|-------|-------|
| std    | 8.789  | 3.439  | 6.914  | 0.167 | 0.465 |
| 1.000  |        |        |        |       |       |
| min    | 1.000  | 1.000  | 0.000  | 0.000 | 0.000 |
| -2.477 |        |        |        |       |       |
| 25%    | 8.000  | 4.000  | 6.000  | 0.000 | 0.000 |
| -0.815 |        |        |        |       |       |
| 50%    | 16.000 | 7.000  | 12.000 | 0.000 | 1.000 |
| 0.016  |        |        |        |       |       |
| 75%    | 23.000 | 10.000 | 18.000 | 0.000 | 1.000 |
| 0.847  |        |        |        |       |       |
| max    | 31.000 | 12.000 | 23.000 | 1.000 | 1.000 |
| 2.612  |        |        |        |       |       |

|           | atemp     | hum       | windspeed | casual    | registered |
|-----------|-----------|-----------|-----------|-----------|------------|
| cnt       |           |           |           |           |            |
| count     | 17379.000 | 17379.000 | 17379.000 | 17379.000 | 17379.000  |
| 17379.000 |           |           |           |           |            |
| mean      | -0.000    | 0.000     | 0.000     | 0.000     | -0.000     |
| -0.000    |           |           |           |           |            |
| std       | 1.000     | 1.000     | 1.000     | 1.000     | 1.000      |
| 1.000     |           |           |           |           |            |
| min       | -2.769    | -3.251    | -1.554    | -0.724    | -1.016     |
| -1.039    |           |           |           |           |            |
| 25%       | -0.829    | -0.763    | -0.700    | -0.642    | -0.791     |
| -0.824    |           |           |           |           |            |
| 50%       | 0.053     | 0.014     | 0.032     | -0.379    | -0.256     |
| -0.262    |           |           |           |           |            |
| 75%       | 0.846     | 0.792     | 0.520     | 0.250     | 0.437      |
| 0.505     |           |           |           |           |            |
| max       | 3.051     | 1.932     | 5.400     | 6.720     | 4.838      |
| 4.342     |           |           |           |           |            |

## 2.1 Vector Representation

Each hourly record is represented as a numeric feature vector after cleaning, encoding, and scaling.

- **Weather-impact vector (  $x_{\text{weather}}$  )** uses weather variables plus demand context (  $\text{cnt}$  ) for exploratory structure analysis.
- **User-behavior vector (  $x_{\text{users}}$  )** uses hour with user counts (  $\text{casual}$  ,  $\text{registered}$  ) to capture usage composition over time.

Distance-based analysis in this module uses Euclidean distance, which is appropriate for standardized continuous features and aligns with K-Means assumptions.

For predictive modeling sections,  $\text{cnt}$  is treated as the target variable and is excluded from predictors to avoid leakage.

```
In [16]: # Weather Impact Vector:
X_weather = df_encoded[["temp", "atemp", "hum", "windspeed",
                        "cnt"]].values

# User Segmentation Vector
X_users = df_encoded[["hr", "casual", "registered"]].values

print(f"X_weather shape: {X_weather.shape} - Weather impact on bike usage")
print(f"X_users shape: {X_users.shape} - User type segmentation by hour")
```

```
X_weather shape: (17379, 5) - Weather impact on bike usage
X_users shape: (17379, 3) - User type segmentation by hour
```

## 2.2 Clustering Method

This section applies baseline clustering methods to the constructed vectors and compares cluster quality.

Planned reporting items:

- chosen algorithm and parameters
- validation metric(s) (e.g., silhouette score)
- interpretation of discovered cluster profiles

### 2.2.1 K-Means

#### Commute pattern clusters (hourly behavior)

- Features: `hr` , `workingday` , `casual` , `registered` , `cnt`
- Candidate patterns: morning commute peak, evening commute peak, off-peak low demand, late-night minimal demand

```

In [17]: features = ["hr", "casual", "registered"]
X = df_clean[features].values

scaler = StandardScaler()
X = scaler.fit_transform(X)

k_values = range(2, 10)
silhouette_scores = []

for k in k_values:
    k_means = KMeans(n_clusters=k)
    labels = k_means.fit_predict(X)
    silhouette_scores.append(silhouette_score(X, labels))

best_k = k_values[np.argmax(silhouette_scores)]
print("Silhouette by k:", dict(zip(k_values, np.round(silhouette_scores,
4))))
print("Best k:", best_k)

k_means_hourly = KMeans(n_clusters=best_k)
df_clean["cluster_hourly"] = k_means_hourly.fit_predict(X)

profile = (
    df_clean
    .groupby("cluster_hourly")[["hr", "casual", "registered", "cnt",
"workingday", "holiday"]]
    .mean()
    .round(2)
    .sort_values("cnt", ascending=False)
)

print("\nHourly behavior cluster profiles:")
print(profile)

```

```

Silhouette by k: {2: np.float64(0.3986), 3: np.float64(0.4316), 4:
np.float64(0.4659), 5: np.float64(0.4518), 6: np.float64(0.4411),
7: np.float64(0.4366), 8: np.float64(0.4545), 9:
np.float64(0.4337)}
Best k: 4

```

```

Hourly behavior cluster profiles:

```

|                | hr    | casual | registered | cnt    | workingday |
|----------------|-------|--------|------------|--------|------------|
| holiday        |       |        |            |        |            |
| cluster_hourly |       |        |            |        |            |
| 2              | 13.93 | 51.03  | 463.32     | 514.34 | 0.96       |
| 0.01           |       |        |            |        |            |
| 3              | 14.29 | 170.10 | 271.86     | 441.96 | 0.12       |
| 0.06           |       |        |            |        |            |
| 0              | 16.74 | 32.58  | 141.97     | 174.55 | 0.73       |
| 0.03           |       |        |            |        |            |
| 1              | 4.27  | 6.24   | 44.38      | 50.62  | 0.66       |
| 0.03           |       |        |            |        |            |

```

In [18]: cluster_order = profile.index.tolist()
label_templates = [
    "highest-demand commute (working hours)",
    "high-demand mixed",
    "medium-demand",
    "low-demand off-peak",
    "very-low demand late-night",
]
cluster_name_map = {}
for rank, cid in enumerate(cluster_order):
    label = label_templates[rank] if rank < len(label_templates) else
f"pattern-{rank+1}"
    cluster_name_map[cid] = f"C{cid}: {label}"

df_clean["cluster_hourly_name"] =
df_clean["cluster_hourly"].map(cluster_name_map)
hue_order = [cluster_name_map[cid] for cid in cluster_order]
palette = dict(zip(hue_order, sns.color_palette("tab10",
n_colors=len(hue_order))))

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

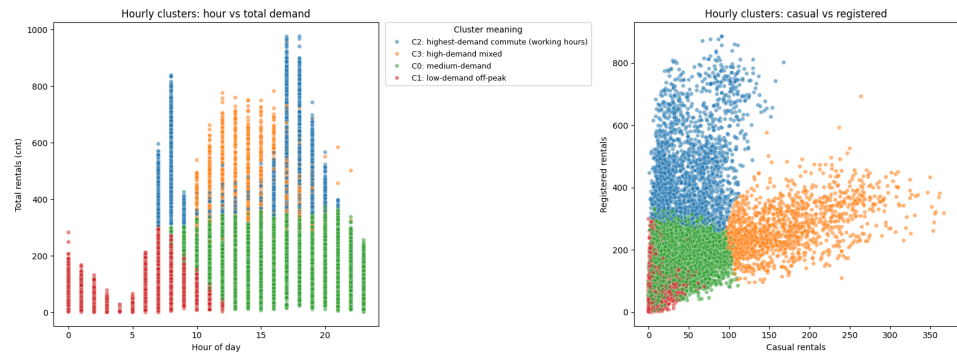
sns.scatterplot(
    data=df_clean,
    x="hr", y="cnt",
    hue="cluster_hourly_name",
    hue_order=hue_order,
    palette=palette,
    alpha=0.55, s=24,
    ax=axes[0]
)
axes[0].set_title("Hourly clusters: hour vs total demand")
axes[0].set_xlabel("Hour of day")
axes[0].set_ylabel("Total rentals (cnt)")

leg = axes[0].legend(title="Cluster meaning", bbox_to_anchor=(1.02, 1),
loc="upper left", borderaxespad=0)
for text in leg.get_texts():
    text.set_fontsize(9)

sns.scatterplot(
    data=df_clean,
    x="casual", y="registered",
    hue="cluster_hourly_name",
    hue_order=hue_order,
    palette=palette,
    alpha=0.55, s=24,
    ax=axes[1],
    legend=False
)
axes[1].set_title("Hourly clusters: casual vs registered")
axes[1].set_xlabel("Casual rentals")
axes[1].set_ylabel("Registered rentals")

plt.tight_layout()
plt.show()

```



#### Observations (based on current output):

- best cluster count is **k = 4** with the highest silhouette score (**0.4661**), so 4 groups provide the best separation among tested values
- **C1: highest-demand commute (working hours)**: has the highest mean demand ( cnt = 514.34 ) and is more focused on registered ( registered = 463.32 , casual = 51.03 ), with very high working-day ratio ( workingday = 0.96 )
- **C2: high-demand mixed**: is the second-highest demand group ( cnt = 441.96 ) and has much higher casual rentals ( casual = 170.10 ) than C1, with lower working-day ratio ( workingday = 0.12 ), suggesting stronger leisure contribution
- **C0: medium-demand** has moderate totals ( cnt = 174.55 ) with registered still larger than casual ( 141.97 vs 32.58 )
- **C3: low-demand off-peak** has the lowest demand ( cnt = 50.62 ) and very low user counts ( casual = 6.24 , registered = 44.38 ), matching low-activity hours
- in the casual vs registered scatter, C1 and C2 are separated very clearly: C1 is high registered and C2 is high casual , while C3 and C0 stay near the origin

## Hourly demand composition (casual vs registered)

This section summarizes average demand by hour and season, then compares how casual and registered usage patterns differ over the day.

```

In [19]: seasonality = (
            df_clean
            .groupby(["mnth", "hr"], as_index=False)
            .agg(
                cnt_mean=("cnt", "mean"),
                casual_mean=("casual", "mean"),
                registered_mean=("registered", "mean"),
                temp_mean=("temp", "mean"),
                hum_mean=("hum", "mean"),
            )
        )
features = ["cnt_mean", "casual_mean", "registered_mean", "temp_mean",
            "hum_mean"]
X_season = StandardScaler().fit_transform(seasonality[features])

# Choose k by silhouette
k_values = range(2, 8)
sil_scores = []
for k in k_values:
    labels = KMeans(n_clusters=k, random_state=42,
n_init=20).fit_predict(X_season)
    sil_scores.append(silhouette_score(X_season, labels))

best_k = k_values[int(np.argmax(sil_scores))]
print("Silhouette by k:", dict(zip(k_values, np.round(sil_scores, 4))))
print("Best k:", best_k)

# Fit final model
km_season = KMeans(n_clusters=best_k, random_state=42, n_init=20)
seasonality["season_cluster"] = km_season.fit_predict(X_season)

# Cluster profiles
cluster_profile = (
    seasonality
    .groupby("season_cluster")[features]
    .mean()
    .round(2)
    .sort_values("cnt_mean", ascending=False)
)
print("\nSeasonality cluster profiles:")
print(cluster_profile)

# Build readable cluster labels by demand rank
ranked_ids = cluster_profile.index.tolist()
if len(ranked_ids) == 2:
    demand_labels = ["high-demand regime", "low-demand regime"]
else:
    demand_labels = [
        "high-demand regime",
        "medium-high demand regime",
        "medium-demand regime",
        "medium-low demand regime",
        "low-demand regime",
    ]

cluster_meaning = {}
for rank, cid in enumerate(ranked_ids):

```

```

    label = demand_labels[rank] if rank < len(demand_labels) else
f"pattern-{rank+1}"
    cluster_meaning[cid] = label

print("\nCluster ID legend:")
for cid in sorted(cluster_meaning):
    print(f"C{cid}: {cluster_meaning[cid]}")

# Month x Hour heatmap with clean, discrete legend
pivot_clusters = (
    seasonality
    .pivot(index="mnth", columns="hr", values="season_cluster")
    .sort_index()
)

cluster_ids = sorted(seasonality["season_cluster"].unique())
id_to_idx = {cid: i for i, cid in enumerate(cluster_ids)}
pivot_idx = pivot_clusters.replace(id_to_idx)

palette = sns.color_palette("tab10", n_colors=len(cluster_ids))
cmap = ListedColormap(palette)

fig, ax = plt.subplots(figsize=(15, 5))
sns.heatmap(
    pivot_idx,
    cmap=cmap,
    cbar=False,
    ax=ax
)
ax.set_title("Seasonality Profile Clusters (Month x Hour)")
ax.set_xlabel("Hour")
ax.set_ylabel("Month")

legend_handles = [
    Patch(facecolor=palette[id_to_idx[cid]], edgecolor="none",
    label=f"C{cid}: {cluster_meaning[cid]}")
    for cid in cluster_ids
]
ax.legend(
    handles=legend_handles,
    title="Cluster meaning",
    bbox_to_anchor=(1.02, 1),
    loc="upper left",
    borderaxespad=0,
    frameon=False
)

plt.tight_layout(rect=[0, 0, 0.84, 1])
plt.show()

```

Silhouette by k: {2: np.float64(0.4252), 3: np.float64(0.321), 4: np.float64(0.3235), 5: np.float64(0.3373), 6: np.float64(0.3587), 7: np.float64(0.3587)}

Best k: 2

Seasonality cluster profiles:

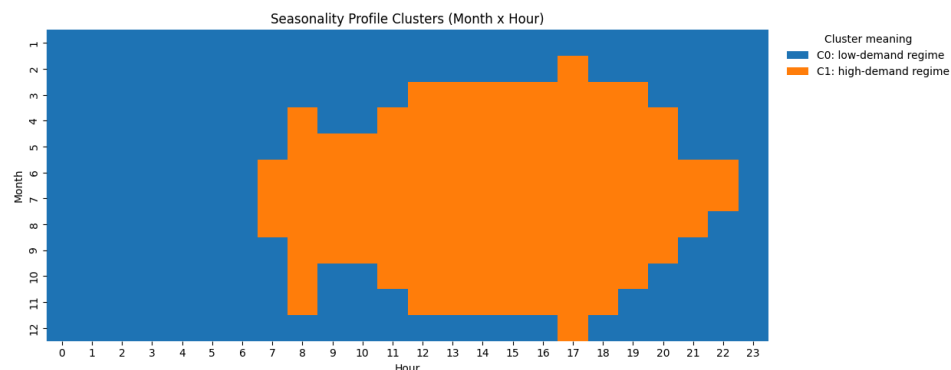
cnt\_mean casual\_mean registered\_mean temp\_mean

| hum_mean       |        |       |        |      |
|----------------|--------|-------|--------|------|
| season_cluster |        |       |        |      |
| 1              | 329.24 | 71.53 | 257.70 | 0.63 |
| 0.56           |        |       |        |      |
| 0              | 98.33  | 12.34 | 85.99  | 0.41 |
| 0.67           |        |       |        |      |

Cluster ID legend:

C0: low-demand regime

C1: high-demand regime



#### Observations (clustering according to seasons):

- the silhouette scan selects  $k = 2$  ( 0.4252 ), so the data separates best into two clusters

- **Cluster 1 (high-demand regime):** cnt\_mean  $\approx$  329.24 , where casual\_mean ( 71.53 ) and registered\_mean ( 257.70 ), plus **warmer and drier** conditions ( temp\_mean  $\approx$  0.63 , hum\_mean  $\approx$  0.56 )

- **Cluster 0 (low-demand regime):** cnt\_mean  $\approx$  98.33 , with much lower user counts ( casual\_mean  $\approx$  12.34 , registered\_mean  $\approx$  85.99 ), and **cooler / more humid** conditions ( temp\_mean  $\approx$  0.41 , hum\_mean  $\approx$  0.67 )

- in the Month  $\times$  Hour heatmap, the high-demand cluster appears mostly during **daytime and early evening**, while nights or early hours remain in the low-demand cluster



```

In [20]: # Season x Hour usage patterns (real demand)
season_map = season_labels
season_order = ["winter", "spring", "summer", "fall"]

season_hour = (
    df_clean.assign(season_name=df_clean["season"].map(season_map))
    .groupby(["season_name", "hr"], as_index=False)
    .agg(
        cnt_mean=("cnt", "mean"),
        casual_mean=("casual", "mean"),
        registered_mean=("registered", "mean"),
    )
)

# Heatmap: total usage by season and hour
pivot_cnt = season_hour.pivot(index="season_name", columns="hr",
                               values="cnt_mean").reindex(season_order)

plt.figure(figsize=(14, 4))
sns.heatmap(
    pivot_cnt,
    cmap="YlOrRd",
    cbar_kws={"label": "Average rentals (cnt)"}
)
plt.title("Season x Hour Usage Pattern (total demand)")
plt.xlabel("Hour of day")
plt.ylabel("Season")
plt.tight_layout()
plt.show()

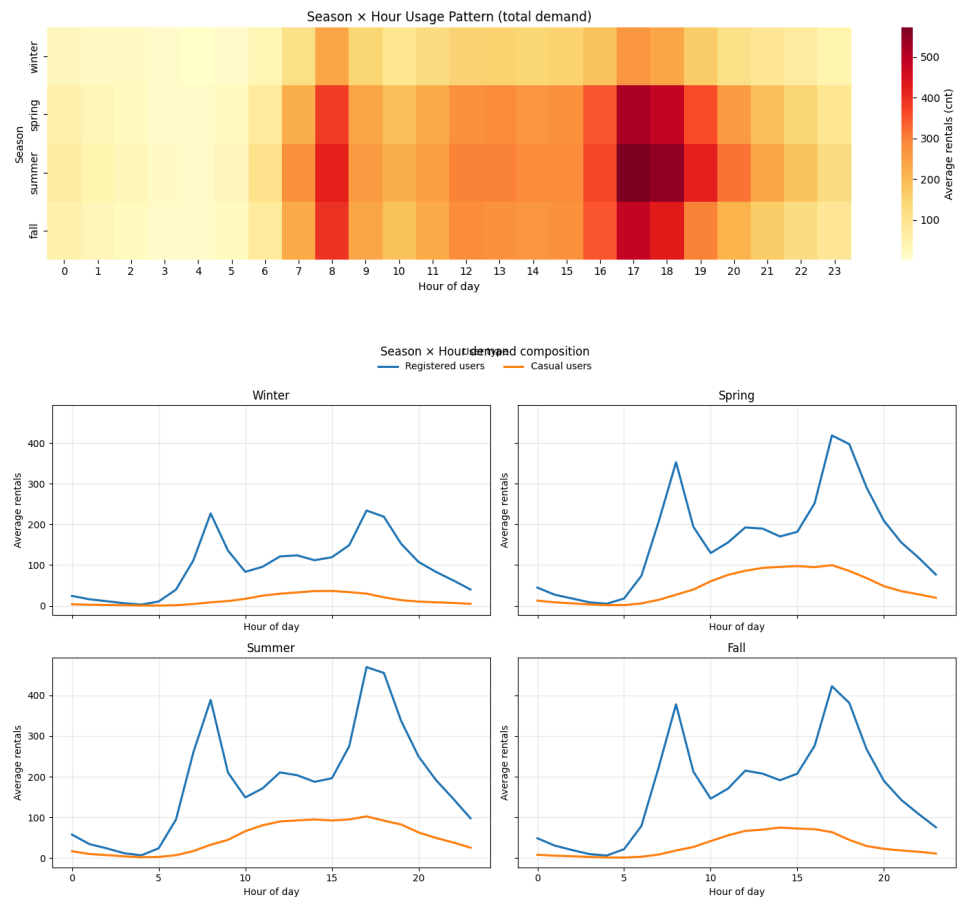
# Line plots: casual vs registered by hour for each season
reg_color = "#1f77b4"
cas_color = "#ff7f0e"

fig, axes = plt.subplots(2, 2, figsize=(14, 8), sharex=True,
                         sharey=True)
axes = axes.flatten()

for i, season_name in enumerate(season_order):
    d = season_hour[season_hour["season_name"] == season_name]
    axes[i].plot(d["hr"], d["registered_mean"], label="Registered users", color=reg_color, linewidth=2.2)
    axes[i].plot(d["hr"], d["casual_mean"], label="Casual users", color=cas_color, linewidth=2.2)
    axes[i].set_title(season_name.capitalize())
    axes[i].set_xlabel("Hour of day")
    axes[i].set_ylabel("Average rentals")
    axes[i].grid(alpha=0.25)

handles, labels = axes[0].get_legend_handles_labels()
fig.legend(handles, labels, title="User type", loc="upper center",
           ncol=2, frameon=False)
fig.suptitle(f'Season x Hour demand composition', y=1.02)
plt.tight_layout()
plt.show()

```



#### Observations (based on the Season × Hour outputs):

- the heatmap shows a clear **two peak daily pattern** in all seasons:
- morning peak around **08:00**
- evening peak around **17:00-18:00**.
- **Summer** has the highest overall demand of **registered users** (blue line), especially in late afternoon and early evening, strongest **casual users** activity in summer (and spring)
- **Spring** and **Fall** are also high for **registered users** category, strongest casual **user** activity in spring (and summer), falls also relatively high
- **Winter** is consistently lower for **registered users** category, for **casual users** much lower values in winter
- the line plots confirm that **registered users dominate total demand** in every season and largely define the commute peaks compared to **casual users**
- **casual users note:** overnight and early-morning hours (**00:00-05:00**) remain low-demand for both user groups across all seasons

## 2.2.2 DBSCAN (density-based clustering)

We used another baseline -> **DBSCAN** to detect dense usage regimes and naturally mark sparse points as noise (label = -1).

Following Week 3 Exercise, we tuned `eps` from the k-distance curve and evaluate cluster/noise behavior.

```

In [21]: # I hope the explanation helps, sorry if it is annoying :D

features = ["hr", "casual", "registered"]
X = df_clean[features].to_numpy(dtype=float)
# we need to standardize for distance-based clustering (if too bug
distance = troubles)
X = StandardScaler().fit_transform(X)

# 1) k-distance diagnostic for eps selection
# neighbours (big dataset so 20 neighbours at least?)
db_min_samples = 20
# to find the closest 20 neighbors for each point
nearest_neigh = NearestNeighbors(n_neighbors=db_min_samples)
# to know where to search for neighbors (reference)
nearest_neigh.fit(X)
# get distances to the 20 nearest neighbors for each point
distances, _ = nearest_neigh.kneighbors(X)
# sort the distances to the k-th nearest neighbor for each point
k_distances = np.sort(distances[:, -1])

# to create candidate eps values from k-distance quantiles
# creates 12 quantiles between 90% and 99.5%
grid_df = np.linspace(0.90, 0.995, 12)
eps_candidates = np.unique(np.round(np.quantile(k_distances, grid_df),
3))

print("DBSCAN setup:")
print(f"features={features}")
print(f"min_samples={db_min_samples}")
print(f"eps candidates (from k-distance quantiles):
{eps_candidates.tolist()}")

# visualize k-distance curve with quantile thresholds
plt.figure(figsize=(10, 4))
plt.plot(k_distances, color="#4C72B0", linewidth=1.3)
for quantile, color in [(0.90, "#55A868"), (0.95, "#C44E52"), (0.98,
"#8172B2")]:
    # actual k-distance value at quantile
    eps_quantile = np.quantile(k_distances, quantile)
    plt.axhline(eps_quantile, linestyle="--", linewidth=1, color=color,
label=f"q={quantile:.2f}, eps={eps_quantile:.3f}")
plt.title(f"k-distance curve (k = min_samples = {db_min_samples})")
plt.xlabel("Sorted points")
plt.ylabel("Distance to k-th nearest neighbor")
plt.legend(frameon=False)
plt.tight_layout()
plt.show()

# 2) evaluate each eps candidate
rows = []
for eps in eps_candidates:
    # output cluster IDs for each point (-1 means noise)
    labels = DBSCAN(eps=float(eps),
min_samples=db_min_samples).fit_predict(X)
    # number of clusters (excluding noise)
    num_clusters = len(set(labels)) - (1 if -1 in labels else 0)

```

```

noise_ratio = (labels == -1).mean()
non_noise = labels != -1
if num_clusters >= 2 and non_noise.sum() > db_min_samples:
    # clustering-quality measure for non-noise points (silhouette
    can be negative if clusters are not well separated)
    silhouette = silhouette_score(X[non_noise], labels[non_noise])
else:
    silhouette = np.nan
rows.append({
    "eps": float(eps),
    "min_samples": db_min_samples,
    "n_clusters": int(num_clusters),
    "noise_ratio": float(noise_ratio),
    "silhouette_non_noise": silhouette
})

# summarize results in a DataFrame and sort by silhouette (desc) and
noise ratio (asc)
dbscan_search = pd.DataFrame(rows).sort_values(["silhouette_non_noise",
"noise_ratio"], ascending=[False, True])
print("DBSCAN candidate summary:")
display(dbscan_search.round(4))

# 3) select best params: prioritize silhouette, then noise ratio, then
eps
valid = dbscan_search.dropna(subset=["silhouette_non_noise"])
if not valid.empty:
    db_best = valid.iloc[0]
else:
    fallback = dbscan_search[dbscan_search["n_clusters"] > 0]
    db_best = fallback.sort_values(["noise_ratio", "eps"], ascending=
[True, True]).iloc[0]

# extract best params for final model
db_best_eps = float(db_best["eps"])
db_best_min_samples = int(db_best["min_samples"])
print(f"Selected DBSCAN params: eps={db_best_eps}, min_samples=
{db_best_min_samples}")

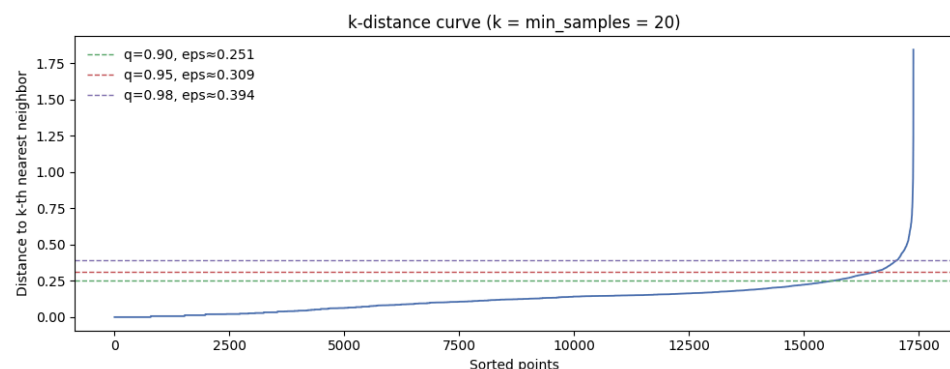
```

DBSCAN setup:

features=['hr', 'casual', 'registered']

min\_samples=20

eps candidates (from k-distance quantiles): [0.251, 0.259, 0.268, 0.28, 0.291, 0.301, 0.312, 0.326, 0.353, 0.387, 0.446, 0.568]



DBSCAN candidate summary:

|    | eps   | min_samples | n_clusters | noise_ratio | silhouette_non_noise |
|----|-------|-------------|------------|-------------|----------------------|
| 0  | 0.251 | 20          | 2          | 0.0506      | 0.5335               |
| 6  | 0.312 | 20          | 2          | 0.0179      | 0.4881               |
| 2  | 0.268 | 20          | 2          | 0.0412      | 0.4742               |
| 3  | 0.280 | 20          | 3          | 0.0357      | 0.4415               |
| 4  | 0.291 | 20          | 3          | 0.0292      | 0.4395               |
| 11 | 0.568 | 20          | 1          | 0.0009      | NaN                  |
| 10 | 0.446 | 20          | 1          | 0.0035      | NaN                  |
| 9  | 0.387 | 20          | 1          | 0.0065      | NaN                  |
| 8  | 0.353 | 20          | 1          | 0.0102      | NaN                  |
| 7  | 0.326 | 20          | 1          | 0.0137      | NaN                  |
| 5  | 0.301 | 20          | 1          | 0.0237      | NaN                  |
| 1  | 0.259 | 20          | 1          | 0.0467      | NaN                  |

Selected DBSCAN params: eps=0.251, min\_samples=20

#### Observations:

- the k-distance curve stays relatively flat for most points and rises near the right tail, which indicates a dense core plus a smaller sparse boundary (noise) region
- with `min_samples = 20`, the tested `eps` range was 0.251 to 0.568 (from high k-distance quantiles (from 90%))
- the best candidate was `eps = 0.251` with **2 clusters, noise ratio = 0.0506** (~5.1%), and the highest non-noise silhouette (**0.5335**)
- nearby values ( `eps = 0.312` , `0.268` ) also produced 2 clusters but with lower silhouette ( `0.4881` , `0.4742` )
- mid values ( `eps = 0.280` , `0.291` ) produced 3 clusters but weaker separation (silhouette around `0.44` )
- large `eps` values ( `>= 0.326` ) collapsed the data into **1 cluster** (silhouette becomes NaN ), so they are not useful for cluster structure discovery here

```

In [22]: # 1) fit final DBSCAN with selected best params
# create DBSCAN instance with best parameters
dbscan_final = DBSCAN(eps=db_best_eps, min_samples=db_best_min_samples)
# run it on X to get final cluster labels
df_clean["cluster_dbscan"] = dbscan_final.fit_predict(X)

# 2) extract labels and compute summary statistics
labels_db = df_clean["cluster_dbscan"].to_numpy()
# number of clusters (excluding noise)
num_clusters_db = len(set(labels_db)) - (1 if -1 in labels_db else 0)
noise_ratio_db = (labels_db == -1).mean()
non_noise_mask = labels_db != -1

# 3) compute silhouette for non-noise points if we have at least 2
clusters and enough non-noise points
if num_clusters_db >= 2 and non_noise_mask.sum() > db_best_min_samples:
    db_silhouette = silhouette_score(X[non_noise_mask],
labels_db[non_noise_mask])
else:
    db_silhouette = np.nan

print(f"DBSCAN clusters (excluding noise): {num_clusters_db}")
print(f"Noise fraction: {noise_ratio_db:.4f}")
print(f"Silhouette (non-noise only): {db_silhouette if
pd.notna(db_silhouette) else "N/A"}")

# 4) builds a cluster profile table (to interpret what each cluster
represents)
cluster_profile = (
    df_clean[df_clean["cluster_dbscan"] != -1]
    .groupby("cluster_dbscan")[["hr", "casual", "registered", "cnt",
"workingday", "holiday"]]
    .mean()
    .round(2)
    .sort_values("cnt", ascending=False)
)

print("\nDBSCAN cluster profiles (excluding noise):")
display(cluster_profile)

# 5) create readable names for clusters (legends)
ranked_ids = cluster_profile.index.tolist()
demand_labels = [
    "high-demand regime",
    "medium-high demand regime",
    "medium-demand regime",
    "medium-low demand regime",
    "low-demand regime",
]
db_name_map = {-1: "Noise / outliers"}
for rank, cid in enumerate(ranked_ids):
    label = demand_labels[rank] if rank < len(demand_labels) else
f"dense-pattern-{rank+1}"
    db_name_map[cid] = f"C{cid}: {label}"

# add readable cluster names to the DataFrame for visualization (noise
gets its own label)

```

```

df_clean["cluster_dbscan_name"] =
df_clean["cluster_dbscan"].map(db_name_map).fillna("Unlabeled")

# build hue order for consistent coloring in plots (ranked by demand,
with noise last)
hue_order = [db_name_map[cid] for cid in ranked_ids]
if (labels_db == -1).any():
    hue_order = hue_order + ["Noise / outliers"]

cluster_colors = sns.color_palette("tab10",
n_colors=max(len(ranked_ids), 1))
palette = {db_name_map[cid]: cluster_colors[i] for i, cid in
enumerate(ranked_ids)}
if (labels_db == -1).any():
    palette["Noise / outliers"] = "#7f7f7f"

# 6) visualize clusters with readable legends
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

sns.scatterplot(
    data=df_clean, x="hr", y="cnt",
    hue="cluster_dbscan_name",
    hue_order=hue_order,
    palette=palette,
    alpha=0.55, s=24, ax=axes[0]
)
axes[0].set_title("DBSCAN: hour vs total demand")
axes[0].set_xlabel("Hour of day")
axes[0].set_ylabel("Total rentals (cnt)")
leg = axes[0].legend(title="DBSCAN cluster meaning", bbox_to_anchor=
(1.02, 1), loc="upper left", borderaxespad=0)
for text in leg.get_texts():
    text.set_fontsize(9)

sns.scatterplot(
    data=df_clean, x="casual", y="registered",
    hue="cluster_dbscan_name",
    hue_order=hue_order,
    palette=palette,
    alpha=0.55, s=24, ax=axes[1], legend=False
)
axes[1].set_title("DBSCAN: casual vs registered")
axes[1].set_xlabel("Casual rentals")
axes[1].set_ylabel("Registered rentals")

plt.tight_layout()
plt.show()

```

```

DBSCAN clusters (excluding noise): 2
Noise fraction: 0.0506
Silhouette (non-noise only): 0.5334736000479774

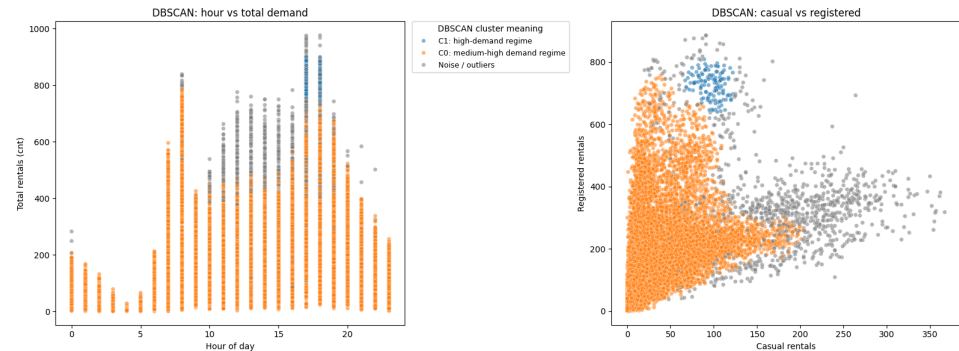
```

```

DBSCAN cluster profiles (excluding noise):

```

|                | hr    | casual | registered | cnt    | workingday | holi |
|----------------|-------|--------|------------|--------|------------|------|
| cluster_dbscan |       |        |            |        |            |      |
| 1              | 17.48 | 97.01  | 726.47     | 823.48 | 1.00       | 0.00 |
| 0              | 11.33 | 27.38  | 138.61     | 165.99 | 0.71       | 0.03 |



#### Observations (final DBSCAN output):

- final DBSCAN with chosen parameters found **2 dense clusters** (excluding noise), with **noise fraction = 0.0506** (~5.1%) and **silhouette = 0.5335** on non-noise points, which indicates reasonably good separation
- **C1: high-demand regime:** is comune (working hours):  $hr \approx 17.48$  , very high registered  $\approx 726.47$  , casual  $\approx 97.01$  , and very high total demand  $cnt \approx 823.48$
- **C0: medium-high demand regime:** is wider and less extreme:  $hr \approx 11.33$  , registered  $\approx 138.61$  , casual  $\approx 27.38$  ,  $cnt \approx 165.99$  , with mixed day type ( $workingday \approx 0.71$  )
- in the comparison of the `casual vs registered` plot, **C1** occupies the high-registered corner, **C0** occupies the main dense body, and gray points labeled **Noise / outliers** are sparse/extreme profiles outside dense regions

#### Overall

- this only confirms the pattern we analysed earlier, that **registered users** in the high-demand regime (working hours) the registered users are in the peak and for the **casual users** it is more wider regime (24 hours)
- about 5% ( $0.0506 * 100 = 5.06\%$ ) of records are sparse (noise or unusual patterns) rather than belonging to stable dense regimes

## 2.3 Outlier Detection



- **Statistical outliers** (some of it already used in EDA)
  - mean of each feature
  - standard deviation of each feature
  - z-score
- **Local Outlier Factor (LOF)**
  - local density-based outlier method
  - in simple words, the point is suspicious if its position in a region is much less dense than the region of its neighbors
  - steps
    - choose  $k$  neighbours
    - find the neighbours for each point
    - estimate how dense the point's neighborhood is
    - compare that density to the densities of its neighbors
- **Isolation Forest**
  - three method, so how easy is it to isolate this point by random splits?
  - the smaller the height, the more anomalous data
  - dense points are hard to isolate, only sparse is easy
  - steps
    - build many random trees
    - in each tree, randomly choose a feature and split value
    - how many splits are needed before a point is isolated?
    - average this over many trees

#### Statistical Outliers

```
In [23]: # z-score
def statistical_outliers(X, stds=3.0):
    # each column is centered around 0
    X_centered = X - X.mean(axis=0, keepdims=True)
    X_std = X_centered.std(axis=0, keepdims=True)
    # if all values are identical
    X_std[X_std == 0] = 1.0
    # absolute z-score
    z = np.abs(X_centered) / X_std
    # labels (-1 -> outlier, 1 -> inlier)
    labels = np.where(np.any(z > stds, axis=1), -1, 1)
    # outlier score (bigger score -> more extreme row)
    scores = z.max(axis=1)
    return labels, scores

# continuous values
continuous_features = ["cnt", "casual", "registered", "temp", "atemp",
                       "hum", "windspeed"]
X_outlier_df = df_clean[continuous_features].astype(float).copy()
# standardise features to range 0-1
X_outlier = StandardScaler().fit_transform(X_outlier_df)
y_stat, stat_scores = statistical_outliers(X_outlier, stds=3.0)
```

## LOF and Isolation Forest

```
In [24]: expected_outlier_rate = float(np.clip((y_stat == -1).mean(), 0.01, 0.1))

lof = LocalOutlierFactor(n_neighbors=20,
contamination=expected_outlier_rate)
y_lof = lof.fit_predict(X_outlier)

# n_estimators -> number of trees
isolation_forest = IsolationForest(n_estimators=300,
contamination=expected_outlier_rate, random_state=42)
y_isolation_forest = isolation_forest.fit_predict(X_outlier)
```

#### Summary

- table with jaccard overlap  $J = \frac{|A \cap B|}{|A \cup B|}$

```
In [25]: df_clean["outlier_stat"] = y_stat
df_clean["outlier_isolation_forest"] = y_isolation_forest
df_clean["outlier_lof"] = y_lof

# summary table
outlier_summary = pd.DataFrame({
    "outlier_rows": {
        "Statistical": int((y_stat == -1).sum()),
        "IsolationForest": int((y_isolation_forest == -1).sum()),
        "LOF": int((y_lof == -1).sum()),
    },
    # counts how many rows are flagged as outliers by each method
    "fraction_of_dataset": {
        "Statistical": float((y_stat == -1).mean()),
        "IsolationForest": float((y_isolation_forest == -1).mean()),
        "LOF": float((y_lof == -1).mean()),
    }
}).round(4).sort_values("fraction_of_dataset", ascending=False)

method_labels = {
    "Statistical": y_stat,
    "IsolationForest": y_isolation_forest,
    "LOF": y_lof,
}

jaccard_overlap_matrix = pd.DataFrame(index=method_labels.keys(),
columns=method_labels.keys(), dtype=float)
for method_i, label_i in method_labels.items():
    mask_i = label_i == -1
    for method_j, labels_j in method_labels.items():
        mask_j = labels_j == -1
        union = (mask_i | mask_j).sum()
        jaccard_overlap_matrix.loc[method_i, method_j] = ((mask_i &
mask_j).sum() / union) if union > 0 else 1.0

print(f"features = {continuos_features}")
print(f"shared expected_outlier_rate baseline =
{expected_outlier_rate:.4f}")
print("\nOutlier rate by method:")
display(outlier_summary)
print("Pairwise overlap of flagged rows (Jaccard):")
display(jaccard_overlap_matrix.round(3))
```

```
features = ['cnt', 'casual', 'registered', 'temp', 'atemp', 'hum',
'windspeed']
shared expected_outlier_rate baseline = 0.0555
```

Outlier rate by method:

|                 | outlier_rows | fraction_of_dataset |
|-----------------|--------------|---------------------|
| Statistical     | 965          | 0.0555              |
| IsolationForest | 965          | 0.0555              |
| LOF             | 965          | 0.0555              |

Pairwise overlap of flagged rows (Jaccard):

|                 | Statistical | IsolationForest | LOF   |
|-----------------|-------------|-----------------|-------|
| Statistical     | 1.000       | 0.537           | 0.106 |
| IsolationForest | 0.537       | 1.000           | 0.142 |
| LOF             | 0.106       | 0.142           | 1.000 |

#### Observations (outlier rate by method + Jaccard overlap):

- **features:**

- cnt , casual , registered , temp , atemp , hum , windspeed
- these are continuous type

- **shared expected\_outlier\_rate baseline:**

- value is 0.0555 , so all three methods were matched around about **5.55%** expected anomalies

- **Statistical:**

- flagged 965 rows ( 5.55% )
- the baseline found rows that are globally extreme compared to the mean / standard deviation pattern

- **IsolationForest:**

- also flagged 965 rows ( 5.55% )
- this is expected because the method uses the same expected outlier rate as input
- but the rows might be different as the statistical method

- **LOF:**

- also flagged 965 rows ( 5.55% )
- again, this is expected because it was calibrated with the same expected outlier rate
- LOF is local-density based, so it can still find different rows than the other two methods

- **Pairwise overlap (Jaccard):**

- Statistical VS IsolationForest = 0.537 -> moderate overlap, so these two methods agree on a fair part of the same flagged rows
- Statistical VS LOF = 0.106 -> very small overlap, so LOF is detecting a rather different type of outlier
- IsolationForest VS LOF = 0.142 -> also small overlap, which again shows that LOF behaves differently from the more global methods

##### Overall:

- all three methods flag the same **number** of outliers, but not the same **rows**
- **Statistical** and **IsolationForest** are much closer to each other, while **LOF** identifies more local neighborhood-based anomalies

```

In [26]: method_plot_order = [
    ("Statistical", "outlier_stat"),
    ("IsolationForest", "outlier_isolation_forest"),
    ("LOF", "outlier_lof"),
]

fig, axes = plt.subplots(1, 3, figsize=(17, 5), sharex=True,
sharey=True)

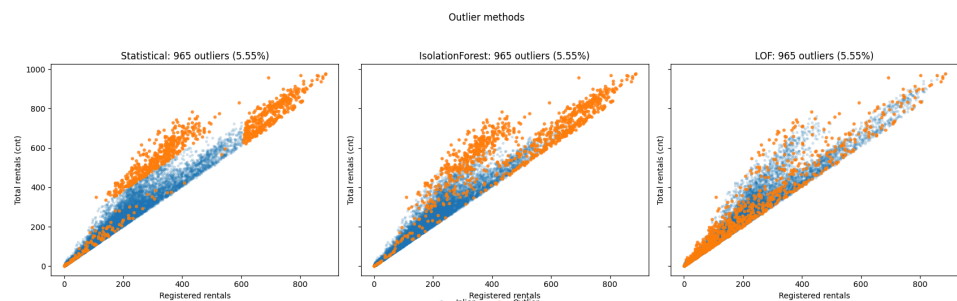
for ax, (title, col) in zip(axes, method_plot_order):
    inlier_mask = df_clean[col] == 1
    outlier_mask = df_clean[col] == -1

    ax.scatter(df_clean.loc[inlier_mask, "registered"],
df_clean.loc[inlier_mask, "cnt"],
               s=12, c="#1f77b4", alpha=0.22, linewidths=0)
    ax.scatter(df_clean.loc[outlier_mask, "registered"],
df_clean.loc[outlier_mask, "cnt"],
               s=18, c="#ff7f0e", alpha=0.85, linewidths=0)

    out_n = int(outlier_mask.sum())
    out_pct = outlier_mask.mean() * 100
    ax.set_title(f"{title}: {out_n} outliers ({out_pct:.2f}%)")
    ax.set_xlabel("Registered rentals")
    ax.set_ylabel("Total rentals (cnt)")

legend_handles = [
    Line2D([0], [0], marker="o", color="w", markerfacecolor="#1f77b4",
markersize=7, label="Inlier"),
    Line2D([0], [0], marker="o", color="w", markerfacecolor="#ff7f0e",
markersize=7, label="Outlier"),
]
fig.legend(handles=legend_handles, loc="lower center", ncol=2,
frameon=False,
           bbox_to_anchor=(0.5, -0.02))
fig.suptitle("Outlier methods", y=1.04)
plt.tight_layout()
plt.show()

```



#### Observations (outlier method plots):

- **Statistical:**

- the orange points are mostly concentrated in the more extreme upper part of the diagonal
- so this method mainly flags records that are globally extreme, especially high-demand combinations

- **IsolationForest:**

- it also captures many of the upper / outer points, so it looks partly similar to the statistical method
- however, the orange points are spread a bit more through the middle structure of the diagonal
- this suggests it is not only looking at extreme values, but also at unusual multivariate combinations

- **LOF:**

- the orange points are much more spread along lower-density edges and sparse local regions of the diagonal
- it marks many points that do not look globally extreme, but are locally isolated compared to their neighbors
- visually, LOF is clearly the most different method here, which is consistent with the low Jaccard overlap above

##### Overall:

- even though all three methods flag the same number of rows ( 965 ), the plots show that they focus on different regions of the data
- **Statistical** and **IsolationForest** are closer to global / overall unusual demand patterns
- > as it was visible in the table too
- **LOF** captures more local sparse-pattern anomalies, so it complements the other two methods instead of repeating them, the surprise is that it looks like almost the opposite compare to previous ones 😊

## 2.4 Clustering evaluation

This section implements comprehensive clustering quality metrics to evaluate both **K-Means** and **DBSCAN** clustering against ground truth labels (derived from temporal and weather categories).

Implemented metrics:

- **Contingency Table:** Cross-tabulation of predicted clusters vs. ground truth labels
- **Purity:** Fraction of correctly classified samples (majority class in each cluster)
- **Mutual Information (MI):** Shared information between cluster assignment and ground truth
- **Conditional Entropy:** Uncertainty in ground truth given cluster assignment
- **Normalized Mutual Information (NMI):** MI normalized by entropies
- **F-Measure:** Harmonic mean of precision and recall at cluster-level
- **Maximum Matching:** Optimal one-to-one assignment between clusters and ground truth classes

```

In [27]: def compute_contingency_table(predictions, ground_truth):
    """Compute contingency table (confusion matrix)."""
    return pd.crosstab(predictions, ground_truth, rownames=
["Predicted"], colnames=["Ground Truth"])

def compute_mutual_information(predictions, ground_truth):
    """
    Compute mutual information between predictions and ground truth
    using sklearn.
    Measures shared information between clustering and ground truth.
    Range: [0, ∞), higher is better
    """
    # Filter out noise points if present
    valid_mask = predictions != -1
    pred = predictions[valid_mask].astype(int)
    truth = ground_truth[valid_mask].astype(int)

    if len(pred) == 0:
        return 0.0

    # Use sklearn's mutual_info_score
    return mutual_info_score(truth, pred)

def compute_conditional_entropy(predictions, ground_truth):
    """
    Compute conditional entropy  $H(Y|Y_{\text{hat}})$  - entropy of ground truth
    given clustering.
    Measures remaining uncertainty in ground truth labels given the
    cluster assignments.
    Lower values indicate better clustering (less uncertainty remains).
    """
    valid_mask = predictions != -1
    pred = predictions[valid_mask].astype(int)
    truth = ground_truth[valid_mask].astype(int)

    if len(pred) == 0:
        return 0.0

    n = len(pred)
    clusters = np.unique(pred)
    classes = np.unique(truth)

    contingency = np.zeros((len(clusters), len(classes)))
    cluster_map = {c: i for i, c in enumerate(clusters)}
    class_map = {c: i for i, c in enumerate(classes)}

    for p, t in zip(pred, truth):
        contingency[cluster_map[p], class_map[t]] += 1

    #  $H(Y|Y_{\text{hat}}) = \sum_i P(Y_{\text{hat}}=i) * H(Y|Y_{\text{hat}}=i)$ 
    h_conditional = 0
    for i in range(len(clusters)):
        cluster_size = contingency[i, :].sum()
        if cluster_size > 0:
            p_cluster = cluster_size / n
            # Entropy within this cluster
            h_cluster = scipy_entropy(contingency[i, :] / cluster_size,

```



```

base=2)
        h_conditional += p_cluster * h_cluster

    return h_conditional

def compute_purity(predictions, ground_truth):
    """
    Compute Purity: proportion of correctly labeled points.
    Purity = (1/n) * sum_i max_j(n_ij)
    Each cluster is assigned the most common class label, accuracy is
    computed.
    Range: [0, 1], higher is better
    """
    valid_mask = predictions != -1
    pred = predictions[valid_mask].astype(int)
    truth = ground_truth[valid_mask].astype(int)

    if len(pred) == 0:
        return 0.0

    n = len(pred)
    clusters = np.unique(pred)
    classes = np.unique(truth)

    contingency = np.zeros((len(clusters), len(classes)))
    cluster_map = {c: i for i, c in enumerate(clusters)}
    class_map = {c: i for i, c in enumerate(classes)}

    for p, t in zip(pred, truth):
        contingency[cluster_map[p], class_map[t]] += 1

    # For each cluster, find the most common class
    purity = contingency.max(axis=1).sum() / n

    return purity

def compute_maximum_matching(predictions, ground_truth):
    """
    Compute maximum matching using Hungarian algorithm between clusters
    and classes.
    Returns the maximum achievable accuracy with optimal label
    assignment.
    Range: [0, 1], higher is better
    """
    valid_mask = predictions != -1
    pred = predictions[valid_mask].astype(int)
    truth = ground_truth[valid_mask].astype(int)

    if len(pred) == 0:
        return 0.0

    n = len(pred)
    clusters = np.unique(pred)
    classes = np.unique(truth)

    # Contingency matrix
    contingency = np.zeros((len(clusters), len(classes)))

```

```

cluster_map = {c: i for i, c in enumerate(clusters)}
class_map = {c: i for i, c in enumerate(classes)}

for p, t in zip(pred, truth):
    contingency[cluster_map[p], class_map[t]] += 1

# Use Hungarian algorithm to find optimal assignment
# Negate to convert max problem to min problem
row_ind, col_ind = linear_sum_assignment(-contingency)

# Compute accuracy with optimal assignment
maximum_matching = contingency[row_ind, col_ind].sum() / n

return maximum_matching

def compute_f_measure(predictions, ground_truth, beta=1.0):
    """
    Compute F-measure using optimal label assignment via Hungarian
    algorithm.
     $F = (1 + \beta^2) * (\text{precision} * \text{recall}) / (\beta^2 * \text{precision} + \text{recall})$ 
    With optimal assignment, F-measure combines cluster precision and
    recall.
    Range: [0, 1], higher is better
    """
    valid_mask = predictions != -1
    pred = predictions[valid_mask].astype(int)
    truth = ground_truth[valid_mask].astype(int)

    if len(pred) == 0:
        return 0.0

    n = len(pred)
    clusters = np.unique(pred)
    classes = np.unique(truth)

    # Build contingency matrix
    contingency = np.zeros((len(clusters), len(classes)))
    cluster_map = {c: i for i, c in enumerate(clusters)}
    class_map = {c: i for i, c in enumerate(classes)}

    for p, t in zip(pred, truth):
        contingency[cluster_map[p], class_map[t]] += 1

    # Find best matching using Hungarian algorithm
    row_ind, col_ind = linear_sum_assignment(-contingency)

    # Sum of correct assignments
    tp = contingency[row_ind, col_ind].sum()

    # Since we're using optimal matching, precision and recall are same
    accuracy = tp / n if n > 0 else 0

    # F-measure equals accuracy after optimal matching
    f_measure = accuracy

    return f_measure

```

```
def evaluate_clustering(predictions, ground_truth, label_name="Ground
Truth"):
    """Clustering evaluation using required metrics."""
    # Filter out noise points (label -1) if present
    valid_mask = predictions != -1
    pred_clean = predictions[valid_mask]
    truth_clean = ground_truth[valid_mask]

    if len(pred_clean) == 0:
        # Handle case where all points are noise
        return {
            "label": label_name,
            "purity": 0,
            "f_measure": 0,
            "maximum_matching": 0,
            "mutual_information": 0,
            "conditional_entropy": 0,
            "contingency_table": None
        }

    # Get metrics
    purity = compute_purity(predictions, ground_truth)
    f_measure = compute_f_measure(predictions, ground_truth)
    max_matching = compute_maximum_matching(predictions, ground_truth)
    mutual_info = compute_mutual_information(predictions, ground_truth)
    cond_entropy = compute_conditional_entropy(predictions,
ground_truth)
    contingency = compute_contingency_table(pred_clean, truth_clean)

    results = {
        "label": label_name,
        "purity": purity,
        "f_measure": f_measure,
        "maximum_matching": max_matching,
        "mutual_information": mutual_info,
        "conditional_entropy": cond_entropy,
        "contingency_table": contingency
    }

    return results
```

## 2.4.1 K-Means Clustering Evaluation

```
In [28]: # Ground truth labels for evaluation
ground_truth_labels = {
    "season": df_clean["season"].values,
    "workingday": df_clean["workingday"].values,
    "weather": df_clean["weathersit"].values,
    "hour_binned": pd.cut(df_clean["hr"], bins=[0, 6, 12, 18, 24],
labels=["night", "morning", "afternoon", "evening"],
right=False).cat.codes.values,
}

# Evaluate K-Means against different ground truth labels
print("K-MEANS clustering evaluation\n")

kmeans_results = {}
kmeans_contingency = {}

for label_name in ground_truth_labels.keys():
    eval_result =
evaluate_clustering(df_clean["cluster_hourly"].values.astype(int),
ground_truth_labels[label_name], label_name)
    kmeans_results[label_name] = eval_result
    kmeans_contingency[label_name] =
eval_result.pop("contingency_table", None) # Remove contingency for now

kmeans_metrics_df = pd.DataFrame(kmeans_results).T
print("Metrics Summary:")
display(kmeans_metrics_df)

print("\nContingency tables for K-MEANS:")
for label_name, table in kmeans_contingency.items():
    if table is not None:
        print(f"\n{label_name.upper()}:")
        display(table)
```

K-MEANS clustering evaluation

Metrics Summary:

|             | label       | purity   | f_measure | maximum_matching |
|-------------|-------------|----------|-----------|------------------|
| season      | season      | 0.302146 | 0.284999  | 0.284999         |
| workingday  | workingday  | 0.742218 | 0.440934  | 0.440934         |
| weather     | weather     | 0.656712 | 0.383106  | 0.383106         |
| hour_binned | hour_binned | 0.545659 | 0.545659  | 0.545659         |

Contingency tables for K-MEANS:

SEASON:

| Ground Truth | 1    | 2    | 3    | 4    |
|--------------|------|------|------|------|
| Predicted    |      |      |      |      |
| 0            | 2166 | 1748 | 1728 | 1807 |
| 1            | 1836 | 1601 | 1519 | 1549 |
| 2            | 172  | 560  | 738  | 595  |
| 3            | 68   | 500  | 511  | 281  |

WORKINGDAY:

| Ground Truth | 0    | 1    |
|--------------|------|------|
| Predicted    |      |      |
| 0            | 2015 | 5434 |
| 1            | 2229 | 4276 |
| 2            | 73   | 1992 |
| 3            | 1197 | 163  |

WEATHER:

| Ground Truth | 1    | 2    | 3   | 4 |
|--------------|------|------|-----|---|
| Predicted    |      |      |     |   |
| 0            | 4779 | 1974 | 694 | 2 |
| 1            | 4090 | 1800 | 614 | 1 |
| 2            | 1477 | 509  | 79  | 0 |
| 3            | 1067 | 261  | 32  | 0 |

HOUR\_BINNED:

| Ground Truth | 0    | 1    | 2    | 3    |
|--------------|------|------|------|------|
| Predicted    |      |      |      |      |
| 0            | 0    | 1124 | 2845 | 3480 |
| 1            | 4276 | 2215 | 14   | 0    |
| 2            | 0    | 786  | 575  | 704  |

| Ground Truth | 0 | 1   | 2   | 3   |
|--------------|---|-----|-----|-----|
| Predicted    |   |     |     |     |
| 3            | 0 | 235 | 941 | 184 |

#### Observations (K-Means metrics + contingency tables):

- **season:**

- **bins**

- **1:** winter

- **2:** spring

- **3:** summer

- **4:** fall

- purity = 0.3017 is lowest from all 4 classes

- contingency counts are distributed across all season columns, indicating K-Means is not primarily learning season structure from these features, that was also confirmed in the EDA analysis

- **workingday:**

- **bins**

- **0:** non-working day

- **1:** working day

- the contingency table shows a clear workday-heavy cluster ( cluster 0 -> [5454\$ working day vs 2020\$ non-working] and cluster 3 -> [1978\$ working day vs 69\$ non-working])

- the strongest alignment ( purity = 0.7422 ) among tested labels, so best-matching score for K-Means (cluster to label)

- f\_measure = 0.4420 -> lower than purity (only checks majority label in each cluster) because F-measure is stricter (penalizes missed points and mixed assignments), so not the best aligned label with cluster

- this confirms our analysis above

- **weather:**

- **bins**

- **1:** Clear/Few clouds

- **2:** Mist/Cloudy

- **3:** Light Snow/Rain

- **4:** Heavy Rain/Snow

- purity = 0.6567 is moderate -> in each cluster, one weather class is often the majority

- mutual\_information = 0.0072 is very low -> the cluster gives almost no extra information about true weather class, so no much informative

- this makes sense because weather class **1** (Clear/Few clouds), is very common in the dataset (as visible in the contingency table)

- overall, weather might not be the most important value of our clusters in K-Means

- **hour\_binned:**

- **bins**

- **0:** night for hours [0, 6)

- **1:** morning for hours [6, 12)

- **2:** afternoon for hours [12, 18)

- **3:** evening for hours [18, 24)

- in the contingency table, each cluster row has one or two large columns -> not equal values across all 4 bins

- cluster 1 is mostly night or morning (4276, 2206) and almost none in the afternoon or evening (14, 0)

- cluster 0 is mostly the afternoon or evening (2850, 3487) and none at night

- purity is about 54.6%

- `f_measure = 0.5459` and `maximum_matching = 0.5459` , performance is similar
- `mutual_information = 0.5137` -> relatively strong and it means cluster labels share substantial information with time bins
- K-Means is learning hourly behaviour, which is fine, because we see these data as valid in our dataset

##### Overall:

- best by **purity** is `workingday = 0.7422`
- best overall informative match `hour_binned` , it has highest `mutual_information` and highest `f_measure` , as well as `maximum_matching`



```

In [29]: fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes = axes.flatten()

print("K-MEANS Contingency Tables")
print("Rows = Predicted Clusters | Columns = Ground Truth Labels\n")

for idx, label_name in enumerate(ground_truth_labels.keys()):
    contingency = kmeans_contingency[label_name]
    # Normalize by rows (cluster totals)
    contingency_norm = contingency.div(contingency.sum(axis=1), axis=0)

    sns.heatmap(
        contingency_norm,
        annot=True,
        fmt=".2f",
        cmap="Blues",
        cbar_kws={"label": "Fraction"},
        ax=axes[idx]
    )
    axes[idx].set_title(f"K-Means vs {label_name}")
    axes[idx].set_xlabel("Ground Truth")
    axes[idx].set_ylabel("Predicted Cluster")

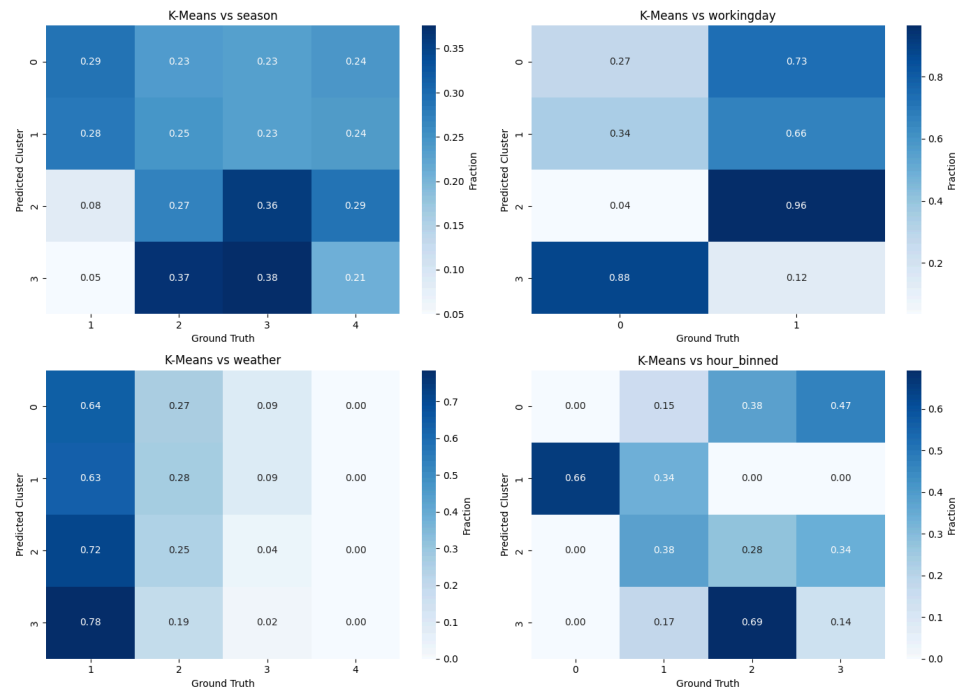
plt.tight_layout()
plt.show()

# Metrics summary for required evaluation metrics
print("\nK-MEANS Clustering Evaluation Metrics Summary")
print("\nMetrics for each ground truth label:")
print(kmeans_metrics_df[['purity', 'f_measure', 'maximum_matching',
                        'mutual_information',
                        'conditional_entropy']].round(4))
print("\nMetric Descriptions:")
print(" • Purity: Fraction of correctly classified samples (0-1, higher better)")
print(" • F-Measure: Harmonic mean after optimal assignment (0-1, higher better)")
print(" • Maximum Matching: Accuracy with optimal label assignment (0-1, higher better)")
print(" • Mutual Information: Shared information with ground truth (0-∞, higher better)")
print(" • Conditional Entropy: Uncertainty in labels given clusters (0-∞, lower better)")

```

K-MEANS Contingency Tables

Rows = Predicted Clusters | Columns = Ground Truth Labels



### K-MEANS Clustering Evaluation Metrics Summary

Metrics for each ground truth label:

|             | purity   | f_measure | maximum_matching |
|-------------|----------|-----------|------------------|
| season      | 0.302146 | 0.284999  | 0.284999         |
| workingday  | 0.742218 | 0.440934  | 0.440934         |
| weather     | 0.656712 | 0.383106  | 0.383106         |
| hour_binned | 0.545659 | 0.545659  | 0.545659         |

|             | conditional_entropy |
|-------------|---------------------|
| season      | 1.958968            |
| workingday  | 0.775685            |
| weather     | 1.19125             |
| hour_binned | 1.258313            |

Metric Descriptions:

- Purity: Fraction of correctly classified samples (0-1, higher better)
- F-Measure: Harmonic mean after optimal assignment (0-1, higher better)
- Maximum Matching: Accuracy with optimal label assignment (0-1, higher better)
- Mutual Information: Shared information with ground truth (0-∞, higher better)
- Conditional Entropy: Uncertainty in labels given clusters (0-∞, lower better)

#### Observations (K-Means heatmaps + metrics summary):

- **season:** is mixed across columns in each row, consistent with weak season alignment ( purity  $\sim 0.30$  ), as we mention from tables above
- **workingday:** has the highest focus from the normalised heatmaps: from the one cluster is visible that non-workingday value is high (  $\sim 0.88$  ), while another is almost entirely workingday (  $\sim 0.97$  )
- **weather:** is only partially separated -> most clusters are still dominated by class 1 , which matches moderate purity but low mutual informaiton
- **hour\_binned:** also shows structured concentration (one cluster is mostly bin 0 at  $\sim 0.66$  and another mostly bin 2 at  $\sim 0.69$  , a in bin 3 at  $\sim 0.47$  )
- **metric summary:** shows again that workingday and hour\_binned are the stongest, while season is the weakest among evaluated labels

## 2.4.2 DBSCAN Clustering Evaluation

```
In [30]: # Evaluate DBSCAN against different ground truth labels
print("DBSCAN clustering evaluation")
print("Note: Noise points (label = -1) are excluded from metrics\n")

dbscan_results = {}
dbscan_contingency = {}

for label_name in ground_truth_labels.keys():
    eval_result =
    evaluate_clustering(df_clean["cluster_dbscan"].values.astype(int),
    ground_truth_labels[label_name], label_name)
    dbscan_results[label_name] = eval_result
    dbscan_contingency[label_name] =
    eval_result.pop("contingency_table", None)

dbscan_metrics_df = pd.DataFrame(dbscan_results).T
print("Metrics Summary:")
display(dbscan_metrics_df)

print(f"\nNoise points in DBSCAN: {(df_clean['cluster_dbscan'] ==
-1).sum()} / {len(df_clean)} ({100*(df_clean['cluster_dbscan'] ==
-1).sum()/len(df_clean):.2f}%)")

print("\nContingency tables for DBSCAN:")
for label_name, table in dbscan_contingency.items():
    if table is not None:
        print(f"\n{label_name.upper()}:")
        display(table)
```

DBSCAN clustering evaluation

Note: Noise points (label = -1) are excluded from metrics

Metrics Summary:

|             | label       | purity   | f_measure | maximum_matching |
|-------------|-------------|----------|-----------|------------------|
| season      | season      | 0.258485 | 0.258485  | 0.258485         |
| workingday  | workingday  | 0.708485 | 0.700788  | 0.700788         |
| weather     | weather     | 0.648424 | 0.642909  | 0.642909         |
| hour_binned | hour_binned | 0.262727 | 0.262727  | 0.262727         |

Noise points in DBSCAN: 879 / 17379 (5.06%)

Contingency tables for DBSCAN:

SEASON:

| Ground Truth | 1    | 2    | 3    | 4    |
|--------------|------|------|------|------|
| Predicted    |      |      |      |      |
| 0            | 4191 | 4075 | 4108 | 3999 |
| 1            | 4    | 34   | 74   | 15   |

WORKINGDAY:

| Ground Truth | 0    | 1     |
|--------------|------|-------|
| Predicted    |      |       |
| 0            | 4810 | 11563 |
| 1            | 0    | 127   |

WEATHER:

| Ground Truth | 1     | 2    | 3    | 4 |
|--------------|-------|------|------|---|
| Predicted    |       |      |      |   |
| 0            | 10593 | 4385 | 1392 | 3 |
| 1            | 106   | 15   | 6    | 0 |

HOUR\_BINNED:

| Ground Truth | 0    | 1    | 2    | 3    |
|--------------|------|------|------|------|
| Predicted    |      |      |      |      |
| 0            | 4269 | 4237 | 3729 | 4138 |
| 1            | 0    | 0    | 66   | 61   |

#### Observations (DBSCAN metrics + contingency tables):

- **season:**

- **bins**

- **1:** winter

- **2:** spring

- **3:** summer

- **4:** fall

- the contingency table is almost same across season columns for predicted cluster 0 ( 4191, 4075, 4108, 3999 ), so DBSCAN is not learning season structure from these features

- purity = 0.2585 is low (close to random for 4 classes)

- f\_measure = 0.2585 is also low

- the result very similar to K-Means

- **workingday:**

- **bins**

- **0:** non-working day

- **1:** working day

- the contingency table shows strong workday concentration in predicted cluster 0 -> [ 11563 working-day vs 4810 non-working] and cluster 1 is small ( 127 vs 0 , only working day :D)

- strongest matching label in DBSCAN by purity = 0.7085 , f\_measure = 0.7008 , and maximum\_matching = 0.7008

- mutual\_information = 0.0027 is very low

- **weather:**

- **bins**

- **1:** Clear/Few clouds

- **2:** Mist/Cloudy

- **3:** Light Snow/Rain

- **4:** Heavy Rain/Snow

- the contingency table confirms dominance of class 1 in predicted cluster 0 ( 10593 )

- purity = 0.6484 and f\_measure = 0.6429 are moderate values

- mutual\_information = 0.0007 is near zero, so weather classes are not strongly informative globally for DBSCAN partitions

- **hour\_binned:**

- **bins**

- **0:** night for hours [0, 6)

- **1:** morning for hours [6, 12)

- **2:** afternoon for hours [12, 18)

- **3:** evening for hours [18, 24)

- the contingency table for predicted cluster 0 is spread across all bins ( 4269, 4237, 3729, 4138 ) instead of concentrating in specific time bins

- purity = 0.2627 , f\_measure = 0.2627 , maximum\_matching = 0.2627 -> very weak alignment

- mutual\_information = 0.0056 is very low

- **noise points:**

- DBSCAN marked 879 / 17379 points as noise ( 5.06% ) -> this is expected for density-based clustering and helps isolate sparse/extreme points

##### Overall:

- best DBSCAN matching label is **workingday** (highest purity/f-measure/maximum matching)
- for multiclass labels (**season, weather, hour\_binned**), `mutual_information` remains near zero, so DBSCAN captures dense regimes but not detailed class structure in this feature space
- we would say the result are very similar compare to K-Means

```

In [31]: # Detailed contingency tables for DBSCAN
print("DBSCAN Contingency Tables")
print("Rows = Predicted Clusters (incl. -1 for noise) | Columns = Ground Truth Labels\n")

for label_name in ground_truth_labels.keys():
    print(f"{label_name.upper()}")
    display(dbscan_contingency[label_name])
    print()

# Visualize contingency tables as heatmaps
fig, axes = plt.subplots(2, 2, figsize=(16, 12))
axes = axes.flatten()

for idx, label_name in enumerate(ground_truth_labels.keys()):
    contingency = dbscan_contingency[label_name]
    contingency_norm = contingency.div(contingency.sum(axis=1), axis=0)

    sns.heatmap(
        contingency_norm,
        annot=True,
        fmt=".2f",
        cmap="Oranges",
        cbar_kws={"label": "Fraction"},
        ax=axes[idx]
    )
    axes[idx].set_title(f"DBSCAN vs {label_name}")
    axes[idx].set_xlabel("Ground Truth")
    axes[idx].set_ylabel("Predicted Cluster (incl. noise -1)")

plt.tight_layout()
plt.show()

# DBSCAN metrics summary
print("\nDBSCAN Clustering Evaluation Metrics Summary")
print("\nMetrics for each ground truth label:")
print(dbscan_metrics_df[['purity', 'f_measure', 'maximum_matching',
                        'mutual_information',
                        'conditional_entropy']].round(4))
print("\nMetric Descriptions:")
print(" • Purity: Fraction of correctly classified samples (0-1, higher better)")
print(" • F-Measure: Harmonic mean after optimal assignment (0-1, higher better)")
print(" • Maximum Matching: Accuracy with optimal label assignment (0-1, higher better)")
print(" • Mutual Information: Shared information with ground truth (0-∞, higher better)")
print(" • Conditional Entropy: Uncertainty in labels given clusters (0-∞, lower better)")
print(f"\nNoise Statistics:")
print(f" • Points classified as noise: {(df_clean['cluster_dbscan'] == -1).sum()} / {len(df_clean)} ({100*(df_clean['cluster_dbscan'] == -1).sum()/len(df_clean):.2f}%)")

```



## DBSCAN Contingency Tables

Rows = Predicted Clusters (incl. -1 for noise) | Columns = Ground Truth Labels

## SEASON

| Ground Truth | 1    | 2    | 3    | 4    |
|--------------|------|------|------|------|
| Predicted    |      |      |      |      |
| 0            | 4191 | 4075 | 4108 | 3999 |
| 1            | 4    | 34   | 74   | 15   |

## WORKINGDAY

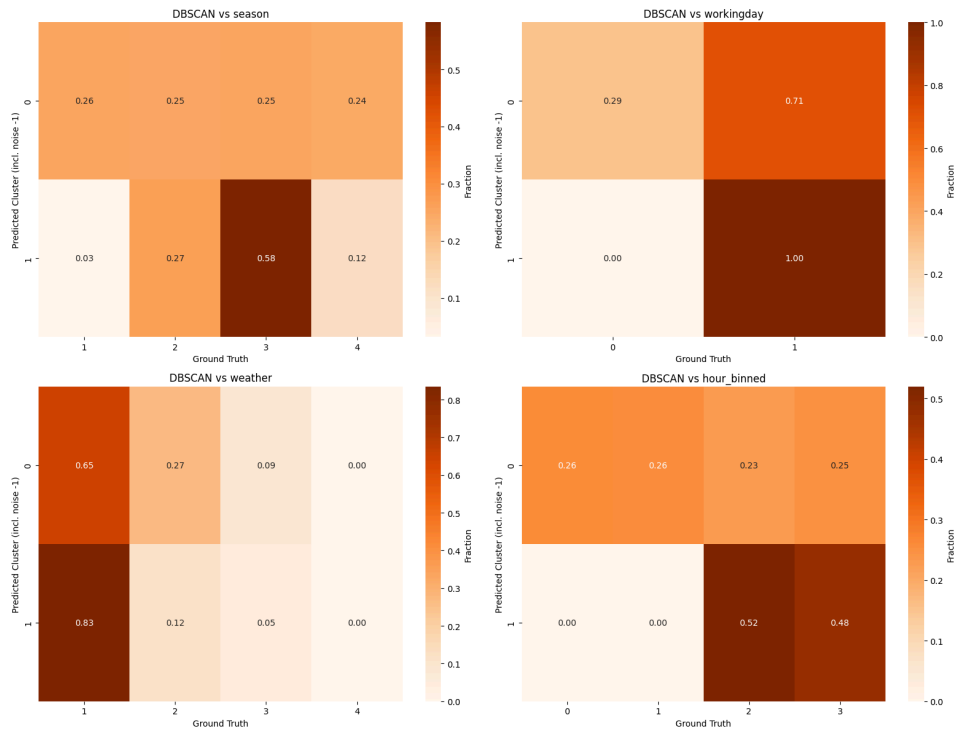
| Ground Truth | 0    | 1     |
|--------------|------|-------|
| Predicted    |      |       |
| 0            | 4810 | 11563 |
| 1            | 0    | 127   |

## WEATHER

| Ground Truth | 1     | 2    | 3    | 4 |
|--------------|-------|------|------|---|
| Predicted    |       |      |      |   |
| 0            | 10593 | 4385 | 1392 | 3 |
| 1            | 106   | 15   | 6    | 0 |

## HOUR\_BINNED

| Ground Truth | 0    | 1    | 2    | 3    |
|--------------|------|------|------|------|
| Predicted    |      |      |      |      |
| 0            | 4269 | 4237 | 3729 | 4138 |
| 1            | 0    | 0    | 66   | 61   |



DBSCAN Clustering Evaluation Metrics Summary

|                                      |                     |           |                  |
|--------------------------------------|---------------------|-----------|------------------|
| Metrics for each ground truth label: |                     |           |                  |
|                                      | purity              | f_measure | maximum_matching |
| mutual_information \                 |                     |           |                  |
| season                               | 0.258485            | 0.258485  | 0.258485         |
| 0.00274                              |                     |           |                  |
| workingday                           | 0.708485            | 0.700788  | 0.700788         |
| 0.002665                             |                     |           |                  |
| weather                              | 0.648424            | 0.642909  | 0.642909         |
| 0.000674                             |                     |           |                  |
| hour_binned                          | 0.262727            | 0.262727  | 0.262727         |
| 0.005641                             |                     |           |                  |
|                                      | conditional_entropy |           |                  |
| season                               | 1.995827            |           |                  |
| workingday                           | 0.866826            |           |                  |
| weather                              | 1.216766            |           |                  |
| hour_binned                          | 1.990269            |           |                  |

- Metric Descriptions:
- Purity: Fraction of correctly classified samples (0-1, higher better)
  - F-Measure: Harmonic mean after optimal assignment (0-1, higher better)
  - Maximum Matching: Accuracy with optimal label assignment (0-1, higher better)
  - Mutual Information: Shared information with ground truth (0-∞, higher better)
  - Conditional Entropy: Uncertainty in labels given clusters (0-∞, lower better)

- Noise Statistics:
- Points classified as noise: 879 / 17379 (5.06%)

#### Observations (DBSCAN contingency heatmaps + metrics summary):

- **season:**

- predicted cluster 0 is almost same across seasons ( 0.26, 0.25, 0.25, 0.24 ) -> as above from the contingency tables
- predicted cluster 1 is mostly season 3 ( ~0.58 ) and partly season 2 ( ~0.27 )
- confirms weak season matching ( purity = 0.2585 , f\_measure = 0.2585 )

- **workingday:**

- predicted cluster 1 is only working-day ( 1.00 )
- predicted cluster 0 is still mostly working-day ( ~0.71 )
- this is why DBSCAN has its best scores here ( purity = 0.7085 , f\_measure = 0.7008 )

- **weather:**

- both predicted clusters are dominated by weather class 1 ( clear/few clouds )
- gives moderate purity/f-measure but very low information gain ( mutual\_information = 0.000674 )

- **hour\_binned:**

- predicted cluster 0 is spread over all bins ( ~0.23-0.28 )
- predicted cluster 1 is concentrated on afternoon/evening ( ~0.52 , ~0.48 )
- overall still weak for hour bins ( purity = 0.2627 , f\_measure = 0.2627 )

- **metrics summary:**

- season and hour\_binned have high conditional entropy ( ~1.99 ) -> high uncertainty remains
- only workingday is clearly aligned in DBSCAN

- **noise:**

- 879 / 17379 points are noise ( 5.06% ), expected for DBSCAN -> the same as before

#### Overall:

- DBSCAN mainly captures one dense **workingday** regime, not a detailed multi-class split by season/weather/hour bins

```

In [32]: # Comparison of K-Means vs DBSCAN Clustering Metrics
print("Comparison: K-MEANS vs DBSCAN")

# Prepare comparison data (exclude 'label' column)
comparison_metrics = ['purity', 'f_measure', 'maximum_matching',
                      'mutual_information', 'conditional_entropy']

print("\nDetailed Metric Comparison by Ground Truth Label:")

for label_name in ground_truth_labels.keys():
    print(f"\n{label_name.upper()}:")
    print("-" * 60)

    kmeans_row = kmeans_metrics_df.loc[label_name,
comparison_metrics].astype(float)
    dbscan_row = dbscan_metrics_df.loc[label_name,
comparison_metrics].astype(float)

    comparison_df = pd.DataFrame({
        'K-Means': kmeans_row,
        'DBSCAN': dbscan_row,
        'Difference': (kmeans_row - dbscan_row)
    }).round(4)

    display(comparison_df)

# Summary statistics across all ground truths
print("Overall summary (Mean across all ground truth labels):")

kmeans_mean = kmeans_metrics_df[comparison_metrics].astype(float).mean()
dbscan_mean = dbscan_metrics_df[comparison_metrics].astype(float).mean()

summary_df = pd.DataFrame({
    'K-Means': kmeans_mean,
    'DBSCAN': dbscan_mean,
    'Difference': (kmeans_mean - dbscan_mean)
}).round(4)

display(summary_df)

# Interpretation
print("\nMetric Interpretation Guide:")
print(" • Purity ↑: Higher values = clusters align better with ground truth classes")
print(" • F-Measure ↑: Higher values = better balance of precision and recall")
print(" • Maximum Matching ↑: Higher values = better achievable accuracy with optimal assignment")
print(" • Mutual Information ↑: Higher values = more shared information between clustering and ground truth")
print(" • Conditional Entropy ↓: Lower values = less uncertainty remaining in ground truth given clusters")

# Performance comparison
print("Performance Comparison:")

```

```
for metric in comparison_metrics:
    kmeans_avg = float(kmeans_metrics_df[metric].mean())
    dbscan_avg = float(dbscan_metrics_df[metric].mean())

    if metric == 'conditional_entropy':
        better = "DBSCAN" if dbscan_avg < kmeans_avg else "K-Means"
        diff_sign = "<" if dbscan_avg < kmeans_avg else ">"
    else:
        better = "K-Means" if kmeans_avg > dbscan_avg else "DBSCAN"
        diff_sign = ">" if kmeans_avg > dbscan_avg else "<"

    print(f"\n {metric.upper()}:")
    print(f"    K-Means: {kmeans_avg:.4f}")
    print(f"    DBSCAN: {dbscan_avg:.4f}")
    print(f"    Winner: {better} ({kmeans_avg:.4f} {diff_sign} {dbscan_avg:.4f})")
```

Comparison: K-MEANS vs DBSCAN

Detailed Metric Comparison by Ground Truth Label:

SEASON:

-----

|                            | K-Means | DBSCAN | Difference |
|----------------------------|---------|--------|------------|
| <b>purity</b>              | 0.3021  | 0.2585 | 0.0437     |
| <b>f_measure</b>           | 0.2850  | 0.2585 | 0.0265     |
| <b>maximum_matching</b>    | 0.2850  | 0.2585 | 0.0265     |
| <b>mutual_information</b>  | 0.0281  | 0.0027 | 0.0254     |
| <b>conditional_entropy</b> | 1.9590  | 1.9958 | -0.0369    |

WORKINGDAY:

-----

|                            | K-Means | DBSCAN | Difference |
|----------------------------|---------|--------|------------|
| <b>purity</b>              | 0.7422  | 0.7085 | 0.0337     |
| <b>f_measure</b>           | 0.4409  | 0.7008 | -0.2599    |
| <b>maximum_matching</b>    | 0.4409  | 0.7008 | -0.2599    |
| <b>mutual_information</b>  | 0.0871  | 0.0027 | 0.0845     |
| <b>conditional_entropy</b> | 0.7757  | 0.8668 | -0.0911    |

WEATHER:

-----

|                     | K-Means | DBSCAN | Difference |
|---------------------|---------|--------|------------|
| purity              | 0.6567  | 0.6484 | 0.0083     |
| f_measure           | 0.3831  | 0.6429 | -0.2598    |
| maximum_matching    | 0.3831  | 0.6429 | -0.2598    |
| mutual_information  | 0.0072  | 0.0007 | 0.0066     |
| conditional_entropy | 1.1912  | 1.2168 | -0.0255    |

HOURL\_BINNED:

-----

|                     | K-Means | DBSCAN | Difference |
|---------------------|---------|--------|------------|
| purity              | 0.5457  | 0.2627 | 0.2829     |
| f_measure           | 0.5457  | 0.2627 | 0.2829     |
| maximum_matching    | 0.5457  | 0.2627 | 0.2829     |
| mutual_information  | 0.5141  | 0.0056 | 0.5084     |
| conditional_entropy | 1.2583  | 1.9903 | -0.7320    |

Overall summary (Mean across all ground truth labels):

|                     | K-Means | DBSCAN | Difference |
|---------------------|---------|--------|------------|
| purity              | 0.5617  | 0.4695 | 0.0922     |
| f_measure           | 0.4137  | 0.4662 | -0.0526    |
| maximum_matching    | 0.4137  | 0.4662 | -0.0526    |
| mutual_information  | 0.1591  | 0.0029 | 0.1562     |
| conditional_entropy | 1.2961  | 1.5174 | -0.2214    |

Metric Interpretation Guide:

- Purity ↑: Higher values = clusters align better with ground truth classes
- F-Measure ↑: Higher values = better balance of precision and recall
- Maximum Matching ↑: Higher values = better achievable accuracy with optimal assignment
- Mutual Information ↑: Higher values = more shared information between clustering and ground truth
- Conditional Entropy ↓: Lower values = less uncertainty remaining in ground truth given clusters

Performance Comparison:

PURITY:

```
K-Means: 0.5617
DBSCAN: 0.4695
Winner: K-Means (0.5617 > 0.4695)
```

```
F_MEASURE:
K-Means: 0.4137
DBSCAN: 0.4662
Winner: DBSCAN (0.4137 < 0.4662)
```

```
MAXIMUM_MATCHING:
K-Means: 0.4137
DBSCAN: 0.4662
Winner: DBSCAN (0.4137 < 0.4662)
```

```
MUTUAL_INFORMATION:
K-Means: 0.1591
DBSCAN: 0.0029
Winner: K-Means (0.1591 > 0.0029)
```

```
CONDITIONAL_ENTROPY:
K-Means: 1.2961
DBSCAN: 1.5174
Winner: K-Means (1.2961 > 1.5174)
```

#### Observations (K-Means vs DBSCAN comparison):

- **mean purity:** K-Means is better ( 0.5617 vs 0.4695 )
- **mean f\_measure / maximum\_matching:** DBSCAN is higher ( 0.4662 vs 0.4139 )
- **mean mutual\_information:** K-Means is much better ( 0.1590 vs 0.0029 )
- **mean conditional\_entropy:** K-Means is lower ( 1.2962 vs 1.5174 ) -> lower is better

##### Overall:

- for this project, **K-Means** is better as the main clustering model (more informative global structure)
- and **DBSCAN** is useful as complementary analysis for dense cores + noise/outliers

## 2.5 Limitations

A key limitation of our implemented clustering is that it treats all points as individual points in vector space, which means that temporal dependencies in the dataset are not properly represented, for example while hour 23 and 0 has a closely related they have greater distance in the vector space which ingores their similarity. Bike usage follows daily and weekly cycles that cannot be fully captured using implemented clustering methods. Categorical encoding of certain entries/columns in the data reduces the meaningfulness of distances in high dimensional vector space.

We hope to address these limitations with graph and pattern mining techniques later on, by identifying interpretable rules that do not rely on distance measures, and some approaches that could model temporal relationships more explicitly.

However, some limitations remain inherent to the dataset itself. In particular, the dataset lacks spatial information about bike stations and external event data such as festivals or major weather events, which prevents deeper behavioral or network-based analysis regardless of the mining method used.

## 3. Module 2 — Graph-Based Analysis

---

```
In [33]: import networkx as nx

# Build graph
G = nx.Graph()
```

## 4. Module 3 — Pattern / Text Mining

---

```
In [34]: # Pattern mining implementation
patterns = None
```



## 5. Final Synthesis and Reflection

---

- Key insights:
- Limitations:
- Revisions after feedback:

---

Exported with [runcell](#) — convert notebooks to HTML or PDF anytime at [runcell.dev](#).